

Project Proposal: *Application of Machine Learning in Prediction of Nuclear Mass*



OLLSCOIL NA GAILLIMHE
UNIVERSITY OF GALWAY

Seáic Mac Murchadha ID: 25255946

S.MacMurchadha1@universityofgalway.ie

School of Computer Science

University of Galway

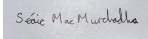
Supervisor(s)

Karl Mason

In partial fulfillment of the requirements for the degree of
MSc in Computer Science (Artificial Intelligence [Online])

Submitted: 30th July 2026

DECLARATION I, Seáic Mac Murchadha, hereby declare that this report, titled “Application of Machine Learning in Prediction of Nuclear Mass”, and the work presented in it are entirely my own except where explicitly stated otherwise in the text, and that this work has not been previously submitted, in part or whole, to any university or institution for any degree, diploma, or other qualification.

Signature: 

Abstract

Research carried out analysed the application of Machine Learning (ML) in refinement of nuclear mass predictions. Various ML techniques were explored, with a diverse range being applicable to this paradigm [1–3]. Cross-pollination opportunities were also explored, with future state-of-the-art level approach potentially coming from adjacent nuclear Physics subfields [4–6]. As code is not available for the current state-of-the-art solutions [1–3], and in order to standardise their task, reengineering of these existing models was carried out in addition to creation of the new solution.

Keywords: ML Application, Nuclear Mass Prediction, ML Refinement, Neural Networks, Theoretical Physics

Contents

List of Acronyms	vi
1 Introduction	1
1.1 Topic	1
1.2 Scope	1
1.3 Method	2
1.4 Layout	3
2 Research Landscape	4
2.1 Motivation	5
2.2 Theoretical background	6
2.2.1 Specific theoretical models	6
2.3 Building ML solutions	7
2.3.1 Magic numbers	8
2.3.2 Feature selection	8
2.4 Uncertainty	9
2.5 ML models	10
2.5.1 State-of-the-art models	12
2.5.2 Proposed model	14
2.6 Interpretability	14

2.7	Data	15
2.8	Conclusion	15
3	Reengineering of LightGBM	16
3.1	LDM	17
3.1.1	5 parameter LDM	17
3.1.2	9 parameter LDM	19
3.2	Selecting Nuclei	21
3.3	Rebuild LightGBM from provided params	22
3.4	Original findings	23
3.5	Change of task	23
3.6	Code	24
4	Reengineering of EKRR	28
4.1	Formulae	28
4.2	Training	30
4.3	Code	32
5	Reengineering of FCNN	36
5.1	Deriving input parameters for the FCNN implementation	36
5.2	Deriving model	39
5.3	Original Results	39
5.4	Additional tuning	40
5.5	Code	41
6	Implementation of PIGNN	48
6.1	Motivation for GNN	48
6.2	GAT	49
6.3	Modelling	49

CONTENTS

6.4	Feature Selection	56
7	Results	59
8	Research Questions	63
9	Conclusion	65
9.1	Progress	65
9.2	Future work	66
	References	73

List of Acronyms

E^{exp} Experimental Binding Energy. 13

E_{LD} Liquid-Drop Model Binding Energy. 13

Adam Adaptive Moment Estimation. 50

AdamW Adaptive Moment Estimation with Decoupled Weight Decay Regularization. 50

AME Atomic Mass Evaluation. 2, 3, 13, 15, 23, 24, 64

ANN Artificial Neural Network. 11

BGP Bayesian Gaussian processes. 9

BMA Bayesian Model Averaging. 9

BNN Bayesian Neural Network. 5, 8–10, 15

CNN Convolutional Neural Network. 1, 14, 48

DFT Density Functional Theory. 9

DL Deep Learning. 2

DZ-10 Duflo-Zuker nuclear mass model with 10 parameters. 23, 57

- DZ-28** Duflo-Zuker nuclear mass model with 28 parameters. 6, 10, 23, 57
- EKRR** Extended Kernel Ridge Regression with odd-even effects. 2, 12, 13, 28, 61, 63, 66
- FCNN** Fully Connected Neural Network. 2, 3, 13, 14, 28, 36, 39–41, 47, 49, 50, 56, 61–63, 66
- FRDM** Finite Range Droplet Model. 6, 7, 9, 11, 23, 57
- GAT** Graph Attention Network. 49
- GCN** Graph Convolutional Network. 49
- GK** Garvey-Kelson. 7, 10
- GK+J** formula based upon Garvey-Kelson mass relations and Jänecke formulas [7]. 10
- GNN** Graph Neural Network. 1–3, 14, 48, 49, 56, 57
- GP** Gaussian Process. 9
- GPR** Gaussian Process Regression. 6, 8, 14
- HFB** Hartree-Fock-Bogoliubov. 9
- knn** k-Nearest Neighbor. 11
- KRR** Kernel Ridge Regression. 12
- LDM** Liquid Drop Model. 6, 7, 10, 11, 13, 17, 19, 23, 61
- LightGBM** Light Gradient Boosting Machine. 2, 12, 14, 16, 17, 19, 22, 28, 61, 63, 66

- MDN** Mixture Density Network. 9, 14
- ML** Machine Learning. ii, 1–4, 6, 7, 9–15, 56, 57, 61, 63
- MSE** Mean Squared Error. 12
- MTL** Multitask Learning. 11, 12
- NLD** Nuclear Level Density. 12
- NN** Neural Network. 13, 66
- PDE** Partial Differential Equation. 12
- PIGNN** Physics Informed Graph Neural Network. 2, 3, 48–50, 61–63, 65, 66
- PIML** Physically Interpretable Machine Learning. 14
- PINN** Physics-Informed Neural Network. 12
- RBF** Radial Basis Function. 10, 12
- RBFOe** Radial Basis Function considering odd-even effects. 10, 12
- RMS** Root Mean Square. 9, 10, 23, 40, 41, 47, 50, 57, 61
- RMSD** Root Mean Square Deviation. 5, 6, 8–13, 23, 29, 39
- SGD** Stochastic Gradient Descent. 40, 50
- SHAP** SHapley Additive exPlanations. 8, 12, 14
- SVR** Support Vector Regression. 8, 14
- WS** Weizsäcker-Skyrme. 6, 57
- WS4** Weizsäcker-Skyrme 4. 6–8, 10, 11, 13, 15, 23, 57, 58, 61

Chapter 1

Introduction

1.1 Topic

Nuclear mass holds an important role, as "the mass of the nucleus must be regarded as one of its basic characteristics" [8]. Whilst theoretical models exist, and have been improving [9–13], ML applications have enabled significant movement in accuracy capabilities by working in tandem with such models [1–3, 14]. By refining existing theoretical models, ML provides improved nuclear data for research into weak-interactions and the astrophysical rapid neutron capture process of nucleosynthesis [8, 15].

1.2 Scope

Whilst a range of ML refinement solutions are already available for nuclear mass predictions, in terms of performant comparisons, state-of-the-art models [1–3] carry the greatest relevance. As Convolutional Neural Network (CNN) [16] provides a matrix capturing neighbouring nuclei it could also provide a relevant comparison to Graph Neural Network (GNN) despite not previously reaching

state-of-the-art level performance.

Extended Kernel Ridge Regression with odd-even effects (EKRR) provides a closed form solution, Light Gradient Boosting Machine (LightGBM) a traditional ML decision tree solution, and Fully Connected Neural Network (FCNN) along with the new GNN solution, Physics Informed Graph Neural Network (PIGNN), provide Deep Learning (DL) solutions, enabling a diverse range of models to be brought into scope.

Experimental data is consistently accumulating over time, with newer datasets becoming available. The standard at a given time is the current iteration of Atomic Mass Evaluation (AME) [17–24]. As AME2016 [21, 22] and AME2020 [23, 24] provide the two latest sets of data, they will be utilised for training and evaluating the models.

Both interpolation and extrapolation are consistently explored [1–3, 14, 25–31] although methodologies of this practice were inconsistent. For nuclear mass predictions, this allows for refinement of currently known nuclei through interpolation, and for exploration of unknown nuclei via extrapolation. A standardised approach of using AME2016 for training, with new nuclei from AME2020 being utilised for extrapolation, and existing nuclei from AME2016 in AME2020 being utilised for interpolation will be taken. Note, despite the nuclei being the same between interpolation and training data, the difference is sufficiently different for a model heavily overfit on AME2016 to not obtain similarly effective scores in AME2020 interpolation which was observed in testing.

1.3 Method

Models were reengineered using parameters mentioned explicitly, or derived from implicit inference. In addition to recreating the models [1–3] from the papers as

per the information available in them, due to the models task being adjusted with a different data selection for both training and prediction, retuning was carried out to provide a fairer comparison between the models, and to the newly proposed GNN.

GNN was also created, with a comparable structure to the FCNN, in terms of layers, in order to provide a fair comparison.

1.4 Layout

The research landscape covers current scientific progress in nuclear mass predictions, both by the theoretical modelling and the ML techniques which build upon them. Various ML techniques are explored, along with distinct paradigms for magic numbers and feature selection. The AME datasets utilised were themselves explored, with justification for prioritisation thereof.

Reengineering of three state-of-the-art models was discussed, with recreation initially, followed by tuning for improved performance. A novel solution, PIGNN, has been planned, and built. Although work is ongoing, implementation of all four models has been carried out, with further tuning and exploration ongoing.

Finally the conclusion discusses current progress, and future direction.

Chapter 2

Research Landscape

In order to begin the project, various papers were reviewed, covering usable datasets, potential and historical ML deployments for both nuclear mass prediction and, to a lesser extent, adjacent nuclear physics applications, which are often highly transferable [2, 4]. As the ML solutions predominantly work in tandem with existing theoretical models, these models were also explored with the applicability to ML refinement as the key focus. For ML applications, cutting-edge solutions were selected within the last few years, some as recent as 2026 [6], with older but still viable ML applications also included, as many papers build upon others, therefore analysing a lineage of research can provide a more holistic view. Older papers, dating back to 1935 [9] were also analysed, as these are the original papers from which the theoretical models refined by ML were discovered.

Category	Papers
ML Applications	[1–3, 14, 16, 25–27, 31–34]
ML Applications - Uncertainty	[28–30]
Theoretical Models	[7, 9–13, 35–38]
Adjacent ML Applications	[4–6]
Future ML Applications	[39–42]
Data	[17–24, 43]
Motivation	[8, 15]
Overview	[44, 45]

Table 2.1: Reviewed Paper Taxonomy

2.1 Motivation

Improving nuclear mass predictions is essential for other branches within nuclear Physics, particularly astrophysics and studies of weak-interactions [8]. Neutron separation energy appears as exponentials for the astrophysical rapid neutron capture process (r process) of nucleosynthesis [15], allowing minor improvements in predictive accuracy to grant a greater refinement for r-process capabilities where half of the heavy elements in nature, from iron to uranium originate [15]. As of 2015 few of the thousands of data points have been experimentally measured for r-process, as these nuclei were far from stability, highlighting the difficulty associated with experimental techniques [15].

Improving prediction accuracy can also expand the usable data, and cross thresholds making more nuclei viable for research. Bayesian Neural Network (BNN) [33] produced the first set of viably precise data for r-process [15] calculations by crossing the 100 keV accuracy threshold. This threshold refers to the Root Mean Square Deviation (RMSD) between the predicted and experimentally observed mass [33]. Crossing accuracy thresholds can enable previously unattainable research to be carried out, demonstrating the cascading effect of these improvements [33].

2.2 Theoretical background

Predicting nuclear mass via modelling is an established area of research, with the Liquid Drop Model (LDM) [9] first being implemented in the 1930s. Since then, research has continued to be carried out into modelling of nuclei, with Finite Range Droplet Model (FRDM) and Weizsäcker-Skyrme (WS) emerging since 1995, and in many cases outperforming existing models [8, 12, 13] with a purely theoretical approach. WS [35] was refined to a model which is retroactively referred to as WS3 [36], with further progress [37] until Weizsäcker-Skyrme 4 (WS4) [12] was finally created.

Whilst WS4 was consistently the optimal model, for both baseline performance, and post-machine learning refinement performance, other models including Duflo-Zuker nuclear mass model with 28 parameters (DZ-28) [11] and FRDM [13] could potentially outperform WS4 due to their differing underlying Physics, should the applied ML solution fill the specific gap of these models more effectively than WS4.

2.2.1 Specific theoretical models

Even the pre-WS4 research was able to achieve RMSD "lower than that achieved with the best of the Duflo-Zuker models." [37] Furthermore, different theoretical models utilise differing underlying Physics, which a given ML solution will be able to improve to various degrees [1, 26]. It is beneficial to compare refinement of multiple different theoretical models when a refinement approach is taken, although a singular theoretical implementation [16] is viable with the WS4 model as a simplified approach. WS4 is often employed as a high-performance theoretical standard for comparison, both in terms of model comparability ("the predictions of the Gaussian Process Regression (GPR)-12 model are comparable to the WS4

model” [26]) and refinement [12] where it provides a theoretical foundation for ML to improve.

Garvey-Kelson (GK) [10] provides a locally fitted theoretical prediction model, in contrast to other global models, as previously discussed (LDM, FRDM, WS4, etc.). GK ”makes very few assumptions about nuclear structure and uses existing experimental masses in an optimal fashion for the prediction of yet unmeasured masses.” [10] Known local masses are leveraged, providing a high degree of accuracy near such nuclei. However, as GK relies on neighbouring values to be known, ideally with the predicted nucleus encapsulated within a 5×5 known nuclei matrix [38], it loses performance as it approaches the edge of the known nuclear chart [10], due to the diminishing amount of known neighbors. This is the issue with employing a local model over a global one.

2.3 Building ML solutions

Whilst existing theoretical models have been progressively improving nuclear mass predictions, as with many subfields within Nuclear Physics [44, 45], ML has provided a substantial improvement in terms of achievable accuracy.

As environments vary between research papers, comparing results obtained in order to cross-evaluate models can prove difficult, with this issue exacerbated via the lack of available code. However, whilst the scripts themselves may not be available, parameters frequently are, allowing for reengineering of the scripts in order to run comparisons. While repeating work may be inefficient, this does provide a viable path to generic reconstructions of a given solution.

2.3.1 Magic numbers

As magic numbers can be difficult to account for, modelling techniques can struggle to handle them. A functional solution is to hard-code the known magic numbers [1, 26], however, this does not scale to extrapolation as it merely encapsulates known magic numbers. Another issue with extrapolation is hidden physics effects, whether they be unaccounted for effects within the known region [14], or potentially from undiscovered effects in the unknown region.

2.3.2 Feature selection

While comparisons between papers are difficult, certain papers do implement comparisons with another paper [26, 27]. One implementation compared Support Vector Regression (SVR) with GPR. An interesting finding of this paper was that both SVR and GPR performance rose as the amount of Physics-based features they used increased.

BNN [14] has been implemented with physics-based features. An interesting observation from results obtained was that despite the expected increase in prediction accuracy across theoretical models, quantified via RMSD, WS4 [12, 25] showed minimal improvements in performance for interpolation. This is likely due to the additional physics-based features added to the model, δ and P for pairing and shell effects respectively, WS4 specifically should largely capture the underlying physics related to these features, therefore applying BNN provides minimal improvement compared to a model for which these features will fill an informative gap. Therefore, whilst physics-based features can provide significant gains in accuracy, they are most likely to do so when the feature is informative. Alternatively, without evaluating the theoretical model itself, systematic techniques can be used via incorporating features incrementally [14, 26], or utilising analysis solutions, such as SHapley Additive exPlanations (SHAP) [1, 26] to discover the

most useful features to implement.

2.4 Uncertainty

Whilst RMSD [1–3, 7, 14, 16, 25–34, 38] has been utilised to evaluate prediction accuracy of nuclear mass. Bayesian [28, 29] methodology, and more recently Mixture Density Network (MDN) [30] have been utilised to quantify the credibility intervals of these predictions, allowing for the range in which a prediction should lie to be quantified. BNN was able to reduce the uncertainty as quantified via Root Mean Square (RMS), although Gaussian Process (GP) fulfilled this function in a more stable and performant manner [28, 29]. Note, BNN and GP (Bayesian Gaussian processes (BGP) [29]) are both Bayesian approaches, enabling uncertainty quantification.

Bayesian Model Averaging (BMA) combines FRDM, Skyrme-Hartree-Fock-Bogoliubov (HFB) and 9 nuclear Density Functional Theory (DFT) rooted theoretical models [29]. Local model averaging from BMA reduces the risk of overfitting [29], whilst simultaneously improving the overall predictive power. Whilst GP and BNN were applied to quantify uncertainty itself [28], BMA [29] was employed to predict the quantity of particle-bound nuclei with a corresponding credibility window, demonstrating versatility of ML approaches.

GP is not suited to extrapolation, as it begins to predict the mean beyond the edges of the known nuclear chart [30], however, the design of MDN enables it to generalise past these limits. MDN [30] provides a probabilistic output with standard deviations, in contrast to the deterministic values commonly derived from neural networks. RMS improvement occurred when Physics-based feature count was increased, decreasing RMS from 3.90 MeV to 0.56 MeV [30], demonstrating the same relationship between informative Physics-based feature count and per-

formance for uncertainty as was previously observed for nuclear mass prediction [26].

2.5 ML models

BNN [25] can be deployed for both global (LDM, WS4, DZ-28) and local (formula based upon Garvey-Kelson mass relations and Jänecke formulas [7] (GK+J)) models. Note, despite GK+J [7] being an improved version of GK, it still inherently lacks extrapolation capabilities, this was quantitatively observed from GK+J only achieving 1010 keV RMSD whilst WS4 was reduced to a RMSD of 340 keV when both had BNN applied [25].

Radial Basis Function (RBF) is another ML approach which has been used for interpolation of known and extrapolation of unknown nuclei. Radial Basis Function considering odd-even effects (RBFoe) [27], an RBF implementation designed to capture the underlying odd-even pattern by default, was deployed and evaluated, as were BNN and pure RBF with comparisons drawn between them. More advanced Physics capturing techniques may be employed elsewhere [39]. However, capturing even the simple odd-even pattern enabled noticeable improvement in predictions as observed between the pure ML implementation of RBF, and the performance of RBF with built-in domain logic with RBFoe [27]. RBFoe has also been compared with BNN, with RBFoe tending to outperform the BNN implementation when either was used in tandem with WS4, the theoretical model providing the lowest RMS when applied with ML techniques. Whilst RBFoe does usually outperform BNN, albeit a simplistic implementation thereof (only one hidden layer, and one output unit were used for the BNN in [27]), it does highlight a use case for employment of an ML selection technique, which would apply the optimal ML solution available for each nucleus to be predicted.

Transfer Learning [42] does not appear to have been applied to predicting nuclear mass so far, however, it has been utilised for predicting nuclear quadrupole deformation parameters [5]. Transfer learning supports tasks with low-quantity, but high-quality training data which aligns with the data [17–24] available for nuclear mass predictions, and adjacent paradigms. Transfer learning mitigates the paucity of experimental data via transferring knowledge from other tasks [42], such as predictions from models, including FRDM [5]. Going forward, this could prove an interesting gap to explore, with ML and theoretical models being often intertwined in their implementation for research [1–3, 14, 29, 34]. Transfer Learning could provide an alternate path to combining theoretical models into the data.

Multitask Learning (MTL) learns multiple tasks in parallel with shared representation [41], multitask learning can be applied to various algorithms including k-Nearest Neighbor (knn), kernel regression and decision trees [41]. The Artificial Neural Network (ANN) methodology was used with MTL for refining LDM [34]. This does leave room for future implementation to be applied with other theoretical models, including the consistently best performing WS4 model [1–3, 14, 25–27], as well as selection of a different algorithm. The primary metric reported was the improvement from MTL, rather than the prediction accuracy itself. While a $\approx 30\text{-}40\%$ reduction in RMSD occurred [34], it did so under what has been the easiest model to improve in other papers [1, 14], LDM. To verify the usefulness of MTL, an evaluation on WS4, the high-performance but difficult-to-improve model would be more informative. Implementing all three tasks simultaneously was outperformed on multiple evaluation metrics by using only two tasks simultaneously: T_B and T_{Sp} (or T_B and T_{Sn}) [34]. Therefore, as with Physics-based features [25, 26], the tasks added must provide a meaningful improvement.

Laws of Physics can be described and implemented into a Physics-Informed Neural Network (PINN) via Partial Differential Equation (PDE) [39]. PINN uses an Mean Squared Error (MSE) loss function, taking account of both experimental values (MSE_u) and the PDE residual term (MSE_f) which penalises non-compliance to these laws of Physics [39]. PINN aligns with the needs of a ML refinement algorithm, which has limited data [17–24], due to its capabilities for ”amplifying the information content of the data” [39]. Code implementation of PINN is available through the original PINN research paper [39]. PINN has been combined with MTL for Nuclear Level Density (NLD) prediction [6], confirming the viability of both PINN [6] and MTL [6, 34] for predicting nuclear observables such as nuclear mass.

2.5.1 State-of-the-art models

LightGBM [1], an advanced decision tree algorithm, at the time of publishing [1, 3] was the state-of-the-art ML solution for theoretical model refinement for nuclear mass interpolation and extrapolation. LightGBM not only provided an effective solution, but also explored the reasoning behind this improvement via analysis tooling, SHAP. By using SHAP [1, 26] researchers have discovered the features which are lacking in various theoretical models. These insights are independently valuable for their contribution to domain knowledge.

Similarly to RBFoe [27] improving RBF via including the odd-even effect, EKRR [2, 4] extends the Kernel Ridge Regression (KRR) ML methodology to account for the effect. In contrast to EKRR, RBF [27] rapidly increases RMSD with extrapolation distance [27], whilst RBFoe overestimates the odd-even effect, for EKRR [2] the RMSD varied ”smoothly with the extrapolation distance” [2], therefore the extrapolation distance issues observed in RBFoe [27] were not present. This is due to RBF methods using the Linear kernel, whilst KRR meth-

ods leverage the Gaussian kernel [2], "it's the combination of Gaussian kernel and ridge regression that leads to the high precision and good extrapolation power of KRR approach" [32]. As was the case with multiple papers [1, 3], EKRR when published was the most precise model for nuclear mass by reducing the RMSD to 128 keV with WS4. The consistently improving models of nuclear mass, especially for ML application, display the rapidly evolving research within this sub-field of nuclear Physics.

The current potential state-of-the-art nuclear mass prediction solution, as of 2025, utilises FCNN [3], however, a directly comparable implementation with competing solutions [2] was not carried out. For FCNN, every neuron in a given layer is connected to every neuron in each adjacent layer. Physics-based features were utilised from the LDM with shell model corrections from v_p and v_n , as well as pairing corrections for the odd-even effect (δ_{np} and $A^{-1/2}$). Further refinement implemented macroscopic-microscopic framework via approximations of the curvature parabola of β_k via refining $A^{-1/2}$ to $g1A^{1/3}$ and $g2A^{-1/3}$, followed by modelling from WS4 and extended accounting for deformation effects. In addition to the initial Experimental Binding Energy (E^{exp}), multi-output was also incorporated for methods II and III, adding the Liquid-Drop Model Binding Energy (E_{LD}) and the difference between E^{exp} and E_{LD} . These steps incrementally increased predictive power [3]. This is similar to previously observed incremental improvements from injecting further relevant Physics into the model [14, 26, 30]. FCNN implementation with the most performant method, method III, achieved RMSD of 52 keV and 122 keV respectively on training and test data, with 191 keV RMSD on extrapolation from AME2016 [21, 22] to AME2020 [23, 24] [3]. The absolute gradient of each feature of the Neural Network (NN) is captured via the normalised gradient method, enabling contributions from features to be assessed individually [3]. FCNN implementation learnt underlying Physics rather

than mere statistical fitting, as demonstrated via sensitivity analysis of independent features including the proton and neutron valence, v_p and v_n respectively, which became dominant in method II and III, indicating learning of the Physics of the shell structure occurred [3].

2.5.2 Proposed model

CNN [16] was previously employed. Although not explicitly stated, due to the geometrically fixed matrices of CNN [16] encapsulating an increasing quantity of unknown nuclei as the matrix approached the edge of the nuclear chart, the accuracy of values predicted diminished. This issue could potentially have been mitigated via using a GNN [40] where only known nuclei would have been included in the model. From currently available material, GNN [40] appears not to have a corresponding research paper currently published for interpolation or extrapolation of nuclear mass.

2.6 Interpretability

Interpretability is essential to ensuring features are informative to the model. For FCNN the normalised gradient method [3] was employed, whilst for SVR, GPR [26] and LightGBM [1] SHAP was utilised. An MDN implementation was carried out, with the novel objective of evaluating feature contributions through Physically Interpretable Machine Learning (PIML) with SHAP [31]. Macroscopic features may dominate in refinement contribution, over quantum effects [31], whilst other models indicate noticeable benefit from quantum features [28]. For ML implementation, analysing individual features during implementation, and verifying each is positively affecting predictions ensures their optimal combination is deployed. Some features have provided minimal improvements such as the

addition of pairing (δ) and shell (P) features to BNN for WS4 [14], as these features' effects are covered via WS4, however, they maintain a positive effect upon the model. Potential redundant or detrimental features can be mitigated against via ablations [3] and feature analysis [1, 26, 31] to ensure their presence is avoided.

2.7 Data

Throughout the various papers [1–3, 5, 7, 14, 16, 25–34, 38] which were explored, AME [17–24] datasets were utilised for training, testing and validation of models. The technique of training on one AME dataset and testing, or extrapolating to another is utilised by both ML [3, 25, 33] and theoretical modelling [7] solutions. Whilst other datasets [43] are available, AME datasets are derived from all available experimental results compiled with a least-squares adjustment applied, providing a master dataset of experimental knowledge for ML implementation to be explored and evaluated upon.

2.8 Conclusion

ML alone cannot "compensate for unknown systematic model deficiencies" [29]. Although ML can improve the reach of nuclear mass accuracy, it still relies upon reliable experimental data on known nuclei. Whilst ML cannot replace experiments, it does provide a highly beneficial component when utilised alongside theoretical (domain) knowledge as evidenced by consistent accuracy improvements when physics-based features were increased [26, 30], including for refinement of predictions, as well as their corresponding uncertainties.

Chapter 3

Reengineering of LightGBM [1]

For the 10 input features of LightGBM (according to table 2 [1]), Z , N and A are already provided via the AME datasets.

- $\frac{N}{Z}$, “Ratio of neutrons to protons”, is readily calculable from Z and N .
- B_LDM , “Theoretical value from the LDM”, was referenced to another paper [46], which revealed the original implementation [47], allowing for recreation to occur.
- N_pair , “Dependence on pair effect”, for odd-odd, odd-even, even-even can be derived from the Z and N variables.
- Z_m , “Shell of the last proton”, can be discovered via the range in which the proton number (Z) resides when such shell capacities are considered.
- N_m , “Shell of the last neutron”, can be discovered via the range in which the neutron number (N) resides when such shell capacities are considered.
- $|Z - m|$, “Distance between proton number and the nearest magic number”, is readily calculable from Z and magic numbers where they are known.

- $|N - m|$, “Distance between neutron number and the nearest magic number”, is readily calculable from N and magic numbers where they are known.

3.1 LDM [46, 47]

LightGBM [1] uses the 9-parameter LDM [46] as an input feature, making it a dependency for implementing LightGBM [1]. This LDM version [46] was recreated as required for LightGBM [1].

3.1.1 5 parameter LDM [9, 47]

Binding Energy:

$$B(A, Z) = a_v A - a_s A^{2/3} - \frac{a_c Z(Z-1)}{A^{1/3}} - \frac{a_{sym}(A-2Z)^2}{A} + \delta$$

where:

$$\delta = \begin{cases} a_p A^{-1/2}, & \text{for even } N\text{-even } Z \\ -a_p A^{-1/2}, & \text{for odd } N\text{-odd } Z \\ 0, & \text{for odd } A \end{cases}$$

- A is the mass number
- Z is the atomic number / proton number for the nucleus
- N is the neutron number for the nucleus
- δ is the pairing energy term
- a_v is the LDM Volume coefficient

- a_s is the LDM Surface coefficient
- a_c is the LDM Coulomb coefficient
- a_{sym} is the LDM Asymmetry coefficient
- a_p is the LDM Pairing coefficient

Atomic mass excess:

$$\Delta M_{A,Z} = Z\Delta m_H + (A - Z)\Delta m_n - a_{el}Z^{2.39} - b_{el}Z^{5.35} - B(A, Z)$$

where:

$$\Delta m_H = m_p + m_e - u = 7.28897050 + a_{el} + b_{el} \text{ MeV}$$

$$\Delta m_n = m_n - u = 8.07131710 \text{ MeV}$$

- m_H is the mass of a hydrogen atom
- m_p is the mass of a proton
- m_n is the mass of a neutron
- m_e is the mass of an electron

a_{el} and b_{el} are the electronic binding energy constants, with $a_{el} = 1.44381 \times 10^{-5} \text{ MeV}$ and $b_{el} = 1.55468 \times 10^{-12} \text{ MeV}$

```

1 def five_param_ldm (
2     A, Z, a_v, a_s, a_c, a_sym, delta
3 ):
4     B = (
5         (a_v * A)

```

```

6      - (a_s * (A**(2/3)))
7      - ((a_c * Z * (Z - 1))
8      / (A ** (1/3)))
9      - ((a_sym * ((A - (2 * Z))**2)) / (A))
10     + delta
11    )
12    return B
13

```

3.1.2 9 parameter LDM [1, 46]

9-parameter LDM was implemented in: “Performance of the Levenberg-Marquardt neural network approach in nuclear mass prediction” [46]. Additionally, it was utilised in “Machine learning the nuclear mass” [1] for LightGBM.

$$E_{LDM}(A, Z) = a_v(1 + \frac{4k_v}{A^2} |T_z| (|T_z| + 1))A + a_s(1 + \frac{4k_s}{A^2} |T_z| (|T_z| + 1))A^{\frac{2}{3}} + a_c \frac{Z^2}{A^{\frac{1}{3}}} + f_p \frac{Z^2}{A} + E_p$$

Where:

E_p is the pairing energy:

$$E_p = \begin{cases} \frac{d_n}{N^{1/3}} + \frac{d_p}{Z^{1/3}} + \frac{d_{np}}{A^{2/3}}, & \text{for } Z \text{ and } N \text{ odd,} \\ \frac{d_p}{Z^{1/3}}, & \text{for } Z \text{ odd, } N \text{ even,} \\ \frac{d_n}{N^{1/3}}, & \text{for } Z \text{ even, } N \text{ odd,} \\ 0, & \text{for } Z \text{ and } N \text{ even.} \end{cases}$$

- T_z is the third component of isospin: $T_z = \frac{1}{2}(Z - N)$

```

1  def nine_param_ldm (A, Z):
2      N = A - Z

```

```
3     abs_Tz = np.abs(N - Z) / 2 # Utilising abs() early to reduce
    repetitive abs() calls throughout code
4
5     a_v = -15.4963
6     a_s = 17.7937
7     kv = -1.8232
8     ks = -2.2593
9     ac = 0.7093
10    fp = -1.2739
11    dn = 4.6919
12    dp = 4.7230
13    dnp = -6.4920
14
15    E_LDM = (
16        (a_v * (1 + ((4 * kv) / (A**2)) * abs_Tz * (abs_Tz + 1)) *
17        A)
18        + (a_s * (1 + ((4 * ks) / (A**2)) * abs_Tz * (abs_Tz + 1))
19        * A**(2/3))
20        + (ac * ((Z**2) / (A**(1/3)))) + (fp * ((Z**2) / A))
21    )
22    # + Pairing Energy
23    if Z % 2:
24        if N % 2:
25            E_LDM += (
26                (dn / (N**(1/3)))
27                + (dp / (Z**(1/3)))
28                + (dnp / (A**(2/3))) # for Z odd and N odd
29            )
30        else:
31            E_LDM += dp / (Z**(1/3)) # for Z odd and N even
32    else:
33        if N % 2:
```

```
33     E_LDM += dn / (N**(1/3)) # for Z even and N odd
34     # for Z even and N even, E_LDM += 0
35
36     # Invert to align with positive binding energies
37     return float(- E_LDM)
38
```

Testing the `nine_param_ldm` on Lead 208:

```
1 print(nine_param_ldm(208, 82)) # Pb 208
```

Returned 1625.90419726084, aligning with expected return value.

3.2 Selecting Nuclei

Only “66” new nuclei of AME2020 were used by the published LightGBM implementation [1] despite 120 being introduced in AME2020 [21–24].

95 of the AME2020 new nuclei were found in the LightGBM supplementary material data [1], this would explain 25 of the overall new nuclei not being included in the 66 new nuclei, but does not explain the further 29 new nuclei that were omitted.

3.3 Rebuild LightGBM from provided params

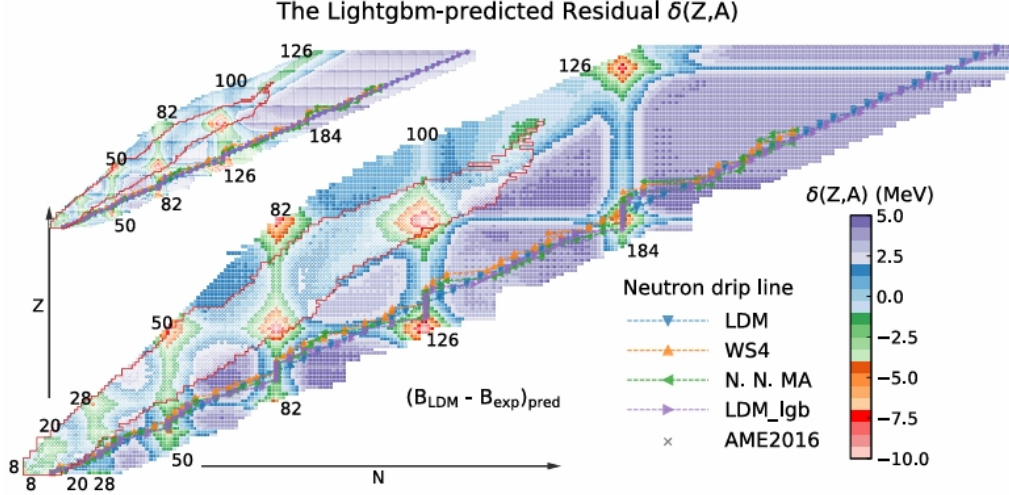


Figure 3.1: Figure 10 of the LightGBM paper [1]

As observed in figure 3.1, a cut off can be observed especially with respect to the Z values, and to a lesser extent with the N values. A Z cutoff of 92, at Uranium (the largest naturally occurring element), will be utilised.

The paper mentioned that nuclei “with relatively large experimental uncertainties in AME2016 were not used”. Although this is with respect to AME2016, this could be tested for AME2020 new nuclei, to decrease the remaining 105 new nuclei down to 66.

3.3 Rebuild LightGBM from provided params [1]

Algorithmic parameters of LightGBM explicitly covered within the paper include: `num_boost_round`, `num_leaves` and `max_depth`.

- `num_boost_round = 50000`
- `num_leaves = 10`

- `max_depth = -1`

3.4 Original findings

From the published paper [1], RMSD achieved with LDM, DZ-28, WS4 and FRDM was 2468, 821, 350, and 778 keV when trained on the existing AME2016 nuclei and extrapolated to AME2020 new nuclei respectively, with LightGBM refinement these respective RMSDs decreased to 452, 320, **222**, and 292 keV. As was the case with the pre-refinement RMSDs, the refined models achieved the greatest accuracy when employing WS4 despite it showing the smallest improvement through refinement. Note, the LightGBM paper references FRDM 2012, not FRDM 1995, and DZ-28, not Duflo-Zuker nuclear mass model with 10 parameters (DZ-10).

3.5 Change of task

While the original LightGBM paper did maintain the same general objective with respect to the predictive outcomes of the model, the details of the original LightGBM paper’s task differ significantly in scope from a broader use of all AME2020 nuclei with respect to both extrapolation and interpolation. As the task has changed, the outcome will as well, targeting all nuclei of AME2020 will encapsulate more unstable nuclei, which will be more difficult to reduce the given RMSD of. Therefore, increased RMS does not necessarily mean diminished results.

AME2020 acts both as validation hold-out data, and tests the extrapolation capabilities of the model. This split will be used for training the model. From the paper, 0.2 was found to be the optimal split of those tested, however the optimal solution for the paper’s task may not be such for other tasks such as those

requiring a larger range of nuclei to be tested. As 20%-80% appears to maintain performance when applied to other tasks, it will be utilised as the train-test split.

Testing with a 0% test could also be meaningful, whilst this creates a severe risk of overfitting, the AME2020 data will still allow for validation to be carried out. `bagging_fraction` and `bagging_freq` enable sampling instead of a train-test split applied across the model, as was used in the paper. Along with `max_bin` these parameters can mitigate overfitting, which is especially useful for reducing the test proportion of the train-test split. Although the LightGBM paper did not provide the code implementation, it did provide parameters for fitting the model, including `num_leaves`, `max_depth` and `verbose`, as well as stating “other parameters are set as their default values of the LightGBM model” therefore tuning parameters such as `max_bin`, `bagging_fraction` or `bagging_freq` would not align with the original implementation, but as the task has changed, so may the tuning of the model to ensure the model is optimised for the given task to which it is being applied.

3.6 Code

```
1 class LightGBM:
2     def __init__(
3         self,
4         test_proportion = 0.2, # AME2020 acts both as validation
                                # hold-out data, and tests the extrapolation capabilities of
                                # the model. This split will be used for training the model.
                                # From the paper, 0.2 was found to be the optimal split of those
                                # tested. However the optimal solution for a given task,
                                # including that of the paper, may not be such for other tasks
                                # such as those requiring a larger range of nuclei to be tested
                                # as is the case in this research.
```

```

5         num_boost_round = 50000,
6         selected_features = ["Z", "N", "A", "N_Z_ratio", "B_ldm",
    "N_pair", "Zm", "Nm", "|Z-m|", "|N-m|"],
7         theoretical_model = "ldm",
8         N_lower_limit = 7,
9         Z_lower_limit = 7,
10        N_upper_limit = 1000, # 1000 upper limit effectively
    disables said limit, as it is outside the range of N and Z
    values.
11        Z_upper_limit = 92,
12    ):
13        self.test_proportion = test_proportion
14        self.num_boost_round = num_boost_round
15        self.selected_features = selected_features
16        self.theoretical_model = theoretical_model
17        self.N_lower_limit = N_lower_limit
18        self.Z_lower_limit = Z_lower_limit
19        self.N_upper_limit = N_upper_limit
20        self.Z_upper_limit = Z_upper_limit
21
22    def get_name(self):
23        return "LightGBM"
24
25    def fit(self, dataset=ame2016_with_features):
26
27        dataset = dataset[(dataset["Z"] > self.Z_lower_limit) & (
    dataset["N"] > self.N_lower_limit) & (dataset["Z"] <= self.
    Z_upper_limit) & (dataset["N"] <= self.N_upper_limit)]
28
29        features = dataset[self.selected_features]
30        target = dataset[self.theoretical_model + "_B_residual"]
31
32        if self.test_proportion > 0:

```

```

33     X_train, X_test, S_train, S_test = train_test_split(
features, target, test_size=self.test_proportion, random_state
=0)
34     else:
35         X_train, S_train = features, target
36         X_test, S_test = X_train, S_train # _test is reused, in
this case as an alias for the training data, therefore
allowing for the training data to be evaluated upon itself for
RMS
37
38     train_data = lgb.Dataset(X_train, label=S_train)
39
40     params = {
41         "num_leaves": 10, # As set in paper
42         "max_depth": -1,
43         "verbose": -1,
44         "seed": 0, # Ensure reproducibility
45         "max_bin": 100 if self.test_proportion == 0 else 250, #
default is 250
46         "bagging_fraction": 0.8 if self.test_proportion == 0 else
1.0, # default is 1.0
47         "bagging_freq": 1 if self.test_proportion == 0 else 0, #
default is 0
48     }
49
50     self.model = lgb.train(
51         params,
52         train_data,
53         num_boost_round=self.num_boost_round
54     )
55
56     self.S = self.model.predict(X_test)
57     self.training_rms = np.sqrt(mean_squared_error(S_test, self

```

```
.S)) * 1000 # *1000 for MeV -> keV
58
59 def predict(self, test_dataset):
60
61     test_dataset = test_dataset[(test_dataset["Z"] > self.
Z_lower_limit) & (test_dataset["N"] > self.N_lower_limit) & (
test_dataset["Z"] <= self.Z_upper_limit) & (test_dataset["N"]
<= self.N_upper_limit)]
62
63     predicted_energy_residuals = self.model.predict(
test_dataset[self.selected_features])
64     predicted_mass_residuals = - predicted_energy_residuals
65     return predicted_mass_residuals
66
67
```

Chapter 4

Reengineering of EKRR [2]

EKRR provides a state-of-the-art solution, as well as a closed-form solution, differentiating it from LightGBM [1] and FCNN [3].

4.1 Formulae

Formulae were provided via the paper [2], and extracted therefrom, enabling recreation of the model.

Extended Kernel Ridge Regression:

$$S(x_j) = \sum_{i=1}^m K(x_j, x_i) \alpha_i + \sum_{i=1}^m K_{oe}(x_j, x_i) \beta_i$$

Euclidean distance between two nuclei:

$$\|x_i - x_j\| = \sqrt{(Z_i - Z_j)^2 + (N_i - N_j)^2}$$

Gaussian Kernel (Describes the similarity between nuclei, decreasing from 1 as distance increases from 0):

$$K(x_j, x_i) = \exp(- || x_i - x_j ||^2 / 2\sigma^2)$$

Odd-Even Kernel:

$$K_{oe}(x_j, x_i) = \delta_{oe}(x_j, x_i) \exp(- || x_i - x_j ||^2 / 2\sigma_{oe}^2)$$

Loss Function:

$$L(\alpha, \beta) = \sum_{i=1}^m [S(x_i) - y(x_i)]^2 + \lambda \alpha^T K \alpha + \lambda_{oe} \beta^T K_{oe} \beta$$

Weights:

$$\beta = \frac{\lambda}{\lambda_{oe}} \alpha$$

$$\alpha = (K + \frac{\lambda}{\lambda_{oe}} K_{oe} + \lambda I)^{-1} y$$

Mass Residual Prediction:

$$S(x_j) = \sum_{i=1}^m K'(x_j, x_i) \alpha_i$$

Remodulated Kernel:

$$K'(x_j, x_i) = K(x_j, x_i) + \frac{\lambda}{\lambda_{oe}} K_{oe}(x_j, x_i)$$

Predicted Mass of a Nucleus:

$$M_{EKRR}(N, Z) = M_{th}(N, Z) + S(N, Z)$$

Hyperparameters were obtained via leave-one-out cross-validation, where RMSD

was minimised:

$$\sigma = 1.25, \lambda = 0.05, \sigma_{oe} = 2.65, \lambda_{oe} = 0.15$$

Where:

- $x_i \equiv (Z_i, N_i)$ corresponding to the locations of training nuclei with respect to the nuclear chart. With x_j for target nuclei.
- m is the number of training nuclei.
- δ_{oe} reflects whether the nuclei have the same parity of proton and neutron number, via ‘1’, or not, via ‘0’. Nuclei with matching Z and N parities should show similar patterns, therefore they should be emphasised when predicting each other.
- σ and σ_{oe} are the Gaussian widths.
- λ and λ_{oe} are hyperparameters determinant of the regularisation strength.
- $y(x_i) = M_{exp}(x_i) - M_{th}(x_i)$ is the true residual, and $S(x_i)$ is the predicted residual.
- I is the identity matrix.
- M_{th} is the theoretical mass.

4.2 Training

Parameters:

$$\sigma = 1.25, \lambda = 0.05, \sigma_{oe} = 2.65, \lambda_{oe} = 0.15$$

Methodology:

$$\begin{bmatrix} Z_1 \\ Z_2 \\ \vdots \\ Z_m \end{bmatrix} \in \mathbb{R}^{(m \times 1)}, \quad \begin{bmatrix} N_1 \\ N_2 \\ \vdots \\ N_m \end{bmatrix} \in \mathbb{R}^{(m \times 1)} \rightarrow \begin{bmatrix} Z_1 & N_1 \\ Z_2 & N_2 \\ \vdots & \vdots \\ Z_m & N_m \end{bmatrix} \in \mathbb{R}^{(m \times 2)} = \text{coords}$$

$$\begin{bmatrix} Z_1 & N_1 \\ Z_2 & N_2 \\ \vdots & \vdots \\ Z_m & N_m \end{bmatrix} \in \mathbb{R}^{(m \times 2)} \rightarrow \begin{bmatrix} |x_1 - x_1|^2 & \cdots & |x_1 - x_m|^2 \\ \vdots & \ddots & \vdots \\ |x_m - x_1|^2 & \cdots & |x_m - x_m|^2 \end{bmatrix} \in \mathbb{R}^{(m \times m)} = \text{euclidean_distance_matrix}$$

$$\begin{bmatrix} |x_1 - x_1|^2 & \cdots & |x_1 - x_m|^2 \\ \vdots & \ddots & \vdots \\ |x_m - x_1|^2 & \cdots & |x_m - x_m|^2 \end{bmatrix} \in \mathbb{R}^{(m \times m)} \rightarrow \begin{bmatrix} e^{-\frac{|x_1 - x_1|^2}{2\sigma^2}} & \cdots & e^{-\frac{|x_1 - x_m|^2}{2\sigma^2}} \\ \vdots & \ddots & \vdots \\ e^{-\frac{|x_m - x_1|^2}{2\sigma^2}} & \cdots & e^{-\frac{|x_m - x_m|^2}{2\sigma^2}} \end{bmatrix} \in \mathbb{R}^{(m \times m)} = K := \text{Gaussian Kernel}$$

$$\begin{bmatrix} Z_1 \bmod 2 \\ Z_2 \bmod 2 \\ \vdots \\ Z_m \bmod 2 \end{bmatrix} \in \{0, 1\}^m, \quad \begin{bmatrix} N_1 \bmod 2 \\ N_2 \bmod 2 \\ \vdots \\ N_m \bmod 2 \end{bmatrix} \in \{0, 1\}^m \rightarrow \begin{bmatrix} \delta_{oe}(x_1, x_1) & \cdots & \delta_{oe}(x_1, x_m) \\ \vdots & \ddots & \vdots \\ \delta_{oe}(x_m, x_1) & \cdots & \delta_{oe}(x_m, x_m) \end{bmatrix} \in \{0, 1\}^{(m \times m)} = \begin{bmatrix} 1 & \cdots & \{0, 1\} & \cdots & 0 \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \{0, 1\} & & \ddots & & \{0, 1\} \\ \vdots & & \ddots & & \vdots \\ 0 & \cdots & \{0, 1\} & \cdots & 1 \end{bmatrix} \in \{0, 1\}^{(m \times m)} = \text{parity_match} := \delta_{oe}$$

$$\begin{bmatrix} \delta_{oe}(x_1, x_1) & \cdots & \delta_{oe}(x_1, x_m) \\ \vdots & \ddots & \vdots \\ \delta_{oe}(x_m, x_1) & \cdots & \delta_{oe}(x_m, x_m) \end{bmatrix} \in \{0, 1\}^{(m \times m)} \circ \begin{bmatrix} e^{-\frac{|x_1 - x_1|^2}{2\sigma_{oe}^2}} & \cdots & e^{-\frac{|x_1 - x_m|^2}{2\sigma_{oe}^2}} \\ \vdots & \ddots & \vdots \\ e^{-\frac{|x_m - x_1|^2}{2\sigma_{oe}^2}} & \cdots & e^{-\frac{|x_m - x_m|^2}{2\sigma_{oe}^2}} \end{bmatrix} \in \mathbb{R}^{(m \times m)} = \begin{bmatrix} \delta_{oe}(x_1, x_1)e^{-\frac{|x_1 - x_1|^2}{2\sigma_{oe}^2}} & \cdots & \delta_{oe}(x_1, x_m)e^{-\frac{|x_1 - x_m|^2}{2\sigma_{oe}^2}} \\ \vdots & \ddots & \vdots \\ \delta_{oe}(x_m, x_1)e^{-\frac{|x_m - x_1|^2}{2\sigma_{oe}^2}} & \cdots & \delta_{oe}(x_m, x_m)e^{-\frac{|x_m - x_m|^2}{2\sigma_{oe}^2}} \end{bmatrix} \in \mathbb{R}^{(m \times m)} = K_{oe} := \text{Odd-Even Kernel}$$

$$K'(x_j, x_i) = K(x_j, x_i) + \frac{\lambda}{\lambda_{oe}} K_{oe}(x_j, x_i)$$

$$\therefore K \in \mathbb{R}^{(m \times m)} + \frac{\lambda}{\lambda_{oe}} \cdot K_{oe} \in \mathbb{R}^{(m \times m)} = \begin{bmatrix} K(x_1, x_1) + \frac{\lambda}{\lambda_{oe}} K_{oe}(x_1, x_1) & \cdots & K(x_1, x_m) + \frac{\lambda}{\lambda_{oe}} K_{oe}(x_1, x_m) \\ \vdots & \ddots & \vdots \\ K(x_m, x_1) + \frac{\lambda}{\lambda_{oe}} K_{oe}(x_m, x_1) & \cdots & K(x_m, x_m) + \frac{\lambda}{\lambda_{oe}} K_{oe}(x_m, x_m) \end{bmatrix} \in \mathbb{R}^{(m \times m)} = \begin{bmatrix} K'(x_1, x_1) & \cdots & K'(x_1, x_m) \\ \vdots & \ddots & \vdots \\ K'(x_m, x_1) & \cdots & K'(x_m, x_m) \end{bmatrix} \in \mathbb{R}^{(m \times m)} = K' = K_{\text{prime}} := \text{Remodulated Kernel}$$

$$(K' + \lambda I) \in \mathbb{R}^{(m \times m)} \cdot \alpha \in \mathbb{R}^{(m \times 1)} = y \in \mathbb{R}^{(m \times 1)} := \left(\begin{bmatrix} K'(x_1, x_1) & \cdots & K'(x_1, x_m) \\ \vdots & \ddots & \vdots \\ K'(x_m, x_1) & \cdots & K'(x_m, x_m) \end{bmatrix} + \lambda \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{bmatrix} \right) \in \mathbb{R}^{(m \times m)} \cdot \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_m \end{bmatrix} \in \mathbb{R}^{(m \times 1)} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix} \in \mathbb{R}^{(m \times 1)}$$

$$\therefore \alpha = (K' + \lambda I)^{-1} y \in \mathbb{R}^{(m \times 1)} = \text{alpha} := \alpha$$

$$\sum_{i=1}^m K'(x_j, x_i) \alpha_i := \begin{bmatrix} K'(x_1, x_1) & \cdots & K'(x_1, x_m) \\ \vdots & \ddots & \vdots \\ K'(x_m, x_1) & \cdots & K'(x_m, x_m) \end{bmatrix} \in \mathbb{R}^{(m \times m)} \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_m \end{bmatrix} \in \mathbb{R}^{(m \times 1)} = \begin{bmatrix} S(x_1) \\ \vdots \\ S(x_m) \end{bmatrix} \in \mathbb{R}^{(m \times 1)} = S := \text{Predicted Mass Residuals}$$

In the original paper [2], leave-one-out cross validation was utilised for hyperparameter tuning. As the tuned hyperparameters were given via the paper [2], these tuned parameters will be used as an initial point. Going forward leave-one-out cross validation could be deployed to recheck the optimality of these parameters.

4.3 Code

```

1 class EKRR:
2     def __init__(
3         self,
4         sigma=1.25, lambda_=0.05, sigma_oe=2.65, lambda_oe=0.15,
5         # Optimised hyperparameters from minimising the RMSD from the
        original implementation [2],
        theoretical_model="ws4",
    )
```

```

6         N_lower_limit=7,
7         Z_lower_limit=7,
8         N_upper_limit = 1000,
9         Z_upper_limit = 1000,
10    ):
11        self.sigma = sigma
12        self.lambda_ = lambda_
13        self.sigma_oe = sigma_oe
14        self.lambda_oe = lambda_oe
15        self.theoretical_model = theoretical_model
16        self.N_lower_limit = N_lower_limit
17        self.Z_lower_limit = Z_lower_limit
18        self.N_upper_limit = N_upper_limit
19        self.Z_upper_limit = Z_upper_limit
20
21    def get_name(self):
22        return "EKRR"
23
24    def fit(self, dataset):
25        torch.manual_seed(0)
26
27        dataset = dataset[(dataset["Z"] > self.Z_lower_limit) & (
dataset["N"] > self.N_lower_limit) & (dataset["Z"] <= self.
Z_upper_limit) & (dataset["N"] <= self.N_upper_limit)]
28
29        Z = dataset["Z"].values
30        N = dataset["N"].values
31        self.Z_parity = dataset["Z_parity"].values
32        self.N_parity = dataset["N_parity"].values
33        y_xi = dataset[self.theoretical_model + "_M_residual"].
values
34
35        # Build kernels

```

```

36     self.coords = np.stack([Z, N], axis=1) # Create a numpy
array of Z and N
37     euclidean_distance_matrix = np.sum(
38         (self.coords[:, np.newaxis] - self.coords[np.newaxis, :])
**2, # Creates a squared Euclidean distance matrix.
39         axis=-1
40     )
41
42     K = np.exp(-euclidean_distance_matrix / (2 * self.sigma**2)
) # Gaussian Kernel
43     parity_match = np.equal.outer(self.Z_parity, self.Z_parity)
& np.equal.outer(self.N_parity, self.N_parity) # True-false
matching parity matrix, true when Z and N parities of nuclei
match.
44     K_oe = parity_match * np.exp(-euclidean_distance_matrix /
(2 * self.sigma_oe**2)) # Odd-Even Kernel
45
46     K_prime = K + (self.lambda_ / self.lambda_oe) * K_oe #
Remodulated Kernel
47     self.alpha = np.linalg.solve(K_prime + self.lambda_ * np.
eye(len(y_xi)), y_xi) # Alpha weight
48
49     self.S = K_prime @ self.alpha # Predicted Mass Residuals
50     self.training_rms = np.sqrt(np.mean((y_xi - self.S)**2)) *
1000 # RMS between the Predicted Mass Residuals and True Mass
Residuals
51
52     def predict(self, test_dataset):
53
54         test_dataset = test_dataset[(test_dataset["Z"] > self.
Z_lower_limit) & (test_dataset["N"] > self.N_lower_limit) & (
test_dataset["Z"] <= self.Z_upper_limit) & (test_dataset["N"]
<= self.N_upper_limit)]

```

```

55
56     Z = test_dataset["Z"].values
57     N = test_dataset["N"].values
58     test_Z_parity = test_dataset["Z_parity"].values
59     test_N_parity = test_dataset["N_parity"].values
60
61     test_coords = np.stack([Z, N], axis=1)
62     euclidean_distance_matrix = np.sum(
63         (test_coords[:, np.newaxis] - self.coords[np.newaxis, :])
64         **2, # "...where xi ≡ (Zi, Ni) are locations of nuclei in the
65             nuclear chart...", with xj taken from the target data to
66             infer S (the predicted mass residual)
67         axis=-1
68     )
69
70     K = np.exp(-euclidean_distance_matrix / (2 * self.sigma**2)
71 )
72
73     parity_match = np.equal.outer(test_Z_parity, self.Z_parity)
74     & np.equal.outer(test_N_parity, self.N_parity)
75
76     K_oe = parity_match * np.exp(-euclidean_distance_matrix /
77         (2 * self.sigma_oe**2))
78
79     K_prime = K + (self.lambda_ / self.lambda_oe) * K_oe
80
81     S = K_prime @ self.alpha
82
83     return S

```

Chapter 5

Reengineering of FCNN [3]

5.1 Deriving input parameters for the FCNN [3] implementation

Input Features:

- Z : Proton number
- A : Mass number
- $A^{2/3}$ reflecting the surface energy
- E_C/a_c representing the Coulomb energy
- I^2A describing symmetry energy
- $|I|$ describing symmetry energy
- $A^{-1/3}$ reflecting the pairing term's contribution
- V_p : Valence proton number
- V_n : Valence neutron number

5.1 Deriving input parameters for the FCNN implementation

- δ_{np} reflecting the pairing term's contribution
- $\prod_{k \in \{2,4,6\}} b_k \beta_k^2$: deformation contribution

Method III *Coulomb term*:

$$E_C = a_c \frac{Z^2}{A^{1/3}} (1 - 0.76 Z^{-2/3})$$

$$\therefore E_C/a_c = \frac{Z^2}{A^{1/3}} (1 - 0.76 Z^{-2/3})$$

Isospin asymmetry:

$$I = \frac{(N - Z)}{A}$$

Valence neutron & proton numbers:

$$V_p = Z - Z_{core}$$

$$V_n = N - N_{core}$$

where:

- V_p and V_n are the valence proton number and valence neutron number respectively.
- Z_{core} and N_{core} are the nearest smaller magic number of the given Z and N respectively.

Deformation contribution

Total binding energy of a nucleus, including deformation and Strutinsky shell corrections:

$$E(A, Z, \beta) = E_{LD}(A, Z) \prod_{k \in \{2,4,6\}} (1 + b_k \beta_k^2) + E'(A, Z, \beta)$$

5.1 Deriving input parameters for the FCNN implementation

Where the $\beta_k (k = 2, 4, 6)$ deformation parabola's curvature is approximated as:

$$b_k = \left(\frac{k}{2}\right)g_1A^{1/3} + \left(\frac{k}{2}\right)^2g_2A^{-1/3}$$

where:

- $g_1 = 0.00895$ for WS (WS4) [36]
- $g_2 = -0.4632$ for WS (WS4) [36]

Summation form, which is utilised as an input feature. This is utilised in theoretical Physics to replace $g_1A^{1/3}$ and $g_2A^{-1/3}$ for deformation contribution:

$$def = \prod_{k \in 2,4,6} (1 + b_k\beta_k^2)$$

Encoded nucleus parity for method III:

$$\delta_{np} = \begin{cases} 2 - |I|, & \text{if } N \text{ and } Z \text{ both even} \\ |I|, & \text{if } N \text{ and } Z \text{ both odd} \\ 1 - |I|, & \text{if } N \text{ is even, } Z \text{ is odd, and } N > Z \\ 1 - |I|, & \text{if } N \text{ is odd, } Z \text{ is even, and } N < Z \\ 1, & \text{if } N \text{ is even, } Z \text{ is odd, and } N < Z \\ 1, & \text{if } N \text{ is odd, } Z \text{ is even, and } N > Z \end{cases}$$

5.2 Deriving model [3]

Whilst the paper states explicitly “We avoided decay mechanisms like Adam’s learning rate (LR) decay or cosine decay, which can cause the optimizer to get stuck in local minima.” It also states “Training was terminated via early stopping at a threshold loss value of 3.80×10^{-4} for Method III...”, with “...a static learning rate of 0.001...”. Although the exact configuration of the neural network cannot be confirmed, as was the case with LightGBM, the general architecture and some specific parameters can be derived from the papers. An Adam optimizer, capable of reaching the threshold value will be deployed, although the paper does not explicitly mention using Adam for Case B Method III, the best performing solution which was trained on AME2016 and extrapolated to AME2020, it does mention not using LR decay schedulers explicitly for this case and method, therefore indicating Adam was utilised. As resultant performance recreation is the objective, rather than exact methodological recreation, the FCNN created here will follow the methodological constraints outlined in the paper, whilst using the best performing solutions discovered to cover those areas not provided through the published material.

5.3 Original Results [3]

With Case B Method III, an RMSD of **191 keV** when *extrapolated* to all the new nuclei of AME2020, and 125 keV on the training data, note this training data only included AME2016 nuclei with experimental errors less than 100 keV as well as their Z and N both above 20. While Case C Method III did achieve better RMSD on test data, said test data was from a train test split rather than extrapolation to new nuclei of AME2020. As Case B Method III used the most rigorous methodology of testing compared to other case methods including Case

C Method III, Case B Method III will be used as the optimal solution for FCNN.

5.4 Additional tuning

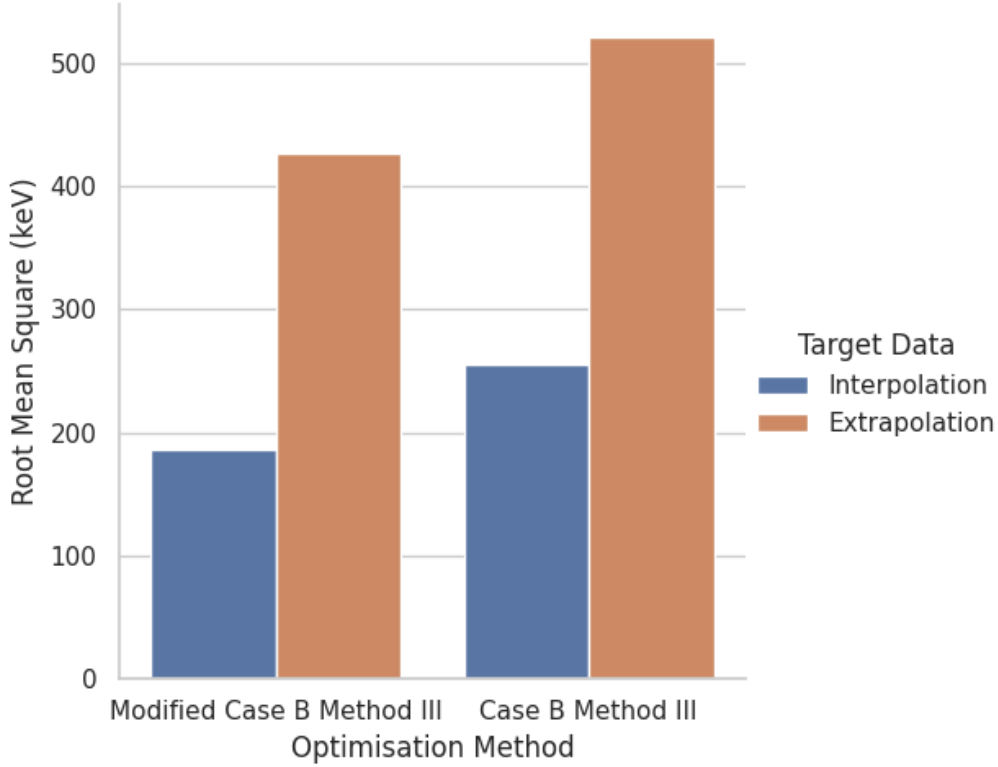


Figure 5.1: FCNN Methods comparison

Whilst the Case B Method III [3] was utilised, it was modified to improve performance on the new task. RMS was measured on the training data, the extrapolation data and the interpolation data, to rigorously inspect performance across epochs. Other methods presented in the paper, such as Case C Method III were tested, as were other potential methods not attempted within the paper, such as applying Stochastic Gradient Descent (SGD) as the optimiser, however, Case B Method III [3] provided the best performance, with modifications. The learn-

ing rate was reduced, and the loss threshold early stop was decreased in value, providing the model greater time to train.

An epoch limit could be selected over a loss threshold for ending iterations, although more testing is required via varying the seed to ensure performance consistency of an epoch limit. As of the runs carried out, 150,000 epochs has consistently been near the minimum extrapolation RMS for FCNN with the modified method.

5.5 Code

```

1  class FCNN:
2      def __init__(
3          self,
4          selected_features,
5          theoretical_model="ws4",
6          N_lower_limit=20,
7          Z_lower_limit=20,
8          N_upper_limit = 1000,  # As no nuclei in the dataset will
          have 1000 N or Z, setting to 1000 disables the upper limit
9          Z_upper_limit = 1000,
10         omit_higher_error_nuclei=False  # Degraded accuracy
11     ):
12         self.selected_features = selected_features
13         self.theoretical_model = theoretical_model
14         self.N_lower_limit = N_lower_limit
15         self.Z_lower_limit = Z_lower_limit
16         self.N_upper_limit = N_upper_limit
17         self.Z_upper_limit = Z_upper_limit
18         self.omit_higher_error_nuclei=omit_higher_error_nuclei
19
20     def get_name(self):

```

```

21     return "FCNN"
22
23     def fit(
24         self,
25         dataset=ame2016_with_features,
26         number_of_epochs=200000,
27         optimisation_method="Modified Case B Method III"
28     ):
29         torch.manual_seed(0)
30
31         self.number_of_epochs = number_of_epochs
32
33         dataset = dataset[
34             (dataset["Z"] > self.Z_lower_limit) & (dataset["N"] >
35             self.N_lower_limit) & (dataset["Z"] <= self.Z_upper_limit) & (
36             dataset["N"] <= self.N_upper_limit)
37             & (dataset["mass_excess_unc"] < 100 if self.
38             omit_higher_error_nuclei else True) # "...only the nuclei
39             with experimental errors less than 100 keV were used."
40         ]
41
42         self.model = nn.Sequential(
43             nn.Linear(len(self.selected_features), 120), # For
44             method III, "...two hidden layers with 120 neurons and 60
45             neurons respectively...".
46             nn.GELU(), # "We selected the GELU activation function
47             in FCNN for its smooth transition between positive and
48             negative values, enhancing gradient flow and enabling neurons
49             to fully learn input features with physical significance. GELU
50             outperforms ReLU and Leaky-ReLU in mitigating gradient
51             vanishing issues and is more accurate and efficient for
52             regression problems than Sigmoid and Tanh."
53             nn.Linear(120, 60),

```

```

42         nn.GELU(),
43         nn.Linear(60, 3) # 3 outputs per Table 1
44     )
45
46     criterion = nn.HuberLoss(delta=1.0) # "...Huber loss
function is used..." "...with  $\delta = 1$ ...".
47
48     self.input_scaler = MinMaxScaler() # "Input features are
scaled to [0, 1] using MinMaxScaler from PyTorch, maintaining
consistency even for binary variables."
49     scaled_input = self.input_scaler.fit_transform(dataset[self
.selected_features])
50     input_tensor = torch.tensor(scaled_input, dtype=torch.
float32)
51
52     self.output_scaler = MinMaxScaler() # "Output labels are
normalized..."
53     scaled_output = self.output_scaler.fit_transform(dataset[["
B_exp_MeV", "B_" + self.theoretical_model, self.
theoretical_model + "_B_residual"]]) # From Table 1
54     output_tensor = torch.tensor(scaled_output, dtype=torch.
float32)
55
56     if optimisation_method in ["Modified Case B Method III", "
Case B Method III"]:
57         if optimisation_method == "Modified Case B Method III":
58             optimizer = torch.optim.Adam(self.model.parameters(),
lr=1e-4)
59
60         if optimisation_method == "Case B Method III": #
Following method described in the paper.
61             optimizer = torch.optim.Adam(self.model.parameters(),
lr=0.001) # "...Method II and III utilized a static learning

```

rate of 0.001..." "We avoided decay mechanisms like Adam's learning rate (LR) decay or cosine decay, which can cause the optimizer to get stuck in local minima." Therefore an Adam implementation with a fixed learning rate of 0.001 will be used.

```

62
63         for epoch in range(self.number_of_epochs): # run 10000
epochs
64             optimizer.zero_grad()
65             prediction = self.model(input_tensor)
66
67             loss = criterion(prediction, output_tensor)
68
69             loss.backward()
70             optimizer.step()
71
72             if loss.item() < 3.80e-4 if optimisation_method == "
Case B Method III" else False: # "Training was terminated via
early stopping at a threshold loss value of...3.80 x 10-4 for
Method III..."
73                 break
74
75             elif optimisation_method == "Case C Method III": #
Following method described in the paper.
76                 optimizer = torch.optim.Adam(self.model.parameters(), lr
=3e-3)
77
78                 for epoch in range(self.number_of_epochs): # Relies upon
stopping threshold
79                     optimizer.zero_grad()
80                     prediction = self.model(input_tensor)
81
82                     loss = criterion(prediction, output_tensor)

```

```

83         loss.backward()
84         optimizer.step()
85
86         if loss.item() > 50e-6: # Switch learning as the loss
87             function decreases, with values provided via table 2 in the
88             FCNN paper.
89                 optimizer.param_groups[0]['lr'] = 3e-3
90             elif loss.item() > 20e-6:
91                 optimizer.param_groups[0]['lr'] = 5e-4
92             else:
93                 optimizer.param_groups[0]['lr'] = 1e-4
94
95             if loss.item() < 2e-5:
96                 break
97
98         else:
99             raise ValueError("Invalid Optimisation method set")
100
101         # Training RMS
102         scaled_predicted_output = self.model(input_tensor).detach()
103         .numpy()
104         predicted_output = self.output_scaler.inverse_transform(
105             scaled_predicted_output)
106         predicted_energy_residuals = predicted_output[:, 2]
107         predicted_mass_residuals = - predicted_energy_residuals
108
109         true_mass_residuals = dataset[self.theoretical_model + "
110             _M_residual"].values
111         self.training_rms = np.sqrt(np.mean((true_mass_residuals -
112             predicted_mass_residuals) ** 2)) * 1000

```

```
110     def predict(self, test_dataset):
111
112         test_dataset = test_dataset[(test_dataset["Z"] > self.
Z_lower_limit) & (test_dataset["N"] > self.N_lower_limit) & (
test_dataset["Z"] <= self.Z_upper_limit) & (test_dataset["N"]
<= self.N_upper_limit)]
113
114         scaled_input = self.input_scaler.transform(test_dataset[
self.selected_features])
115         input_tensor = torch.tensor(scaled_input, dtype=torch.
float32)
116
117         scaled_predicted_output = self.model(input_tensor).detach()
.numpy()
118
119         predicted_output = self.output_scaler.inverse_transform(
scaled_predicted_output)
120
121         predicted_energy_residuals = predicted_output[:, 2]
122         predicted_mass_residuals = - predicted_energy_residuals
123
124         return predicted_mass_residuals
125
126
```


Table 5.1: RMS (keV) of FCNN with modified method across epochs

Epoch	Loss	Training RMS	Interpolation RMS	Extrapolation RMS
10000	0.0004688412	284.080	267.277	564.948
20000	0.0002943035	225.135	235.320	490.419
30000	0.0002060629	187.855	216.814	507.107
40000	0.0001841153	177.466	211.470	510.922
50000	0.0001704853	170.660	208.133	522.687
60000	0.0001580752	164.188	204.758	513.134
70000	0.0001490190	159.288	202.423	500.942
80000	0.0001424943	155.659	200.714	491.903
90000	0.0001371871	152.653	199.195	482.239
100000	0.0001323771	149.903	197.679	468.299
110000	0.0001315137	147.664	196.588	455.710
120000	0.0001241199	145.106	195.007	444.907
130000	0.0001206522	143.091	193.884	439.646
140000	0.0001170702	140.874	192.804	438.333
150000	0.0001140497	138.993	191.766	437.747
160000	0.0001113829	137.312	190.841	438.805
170000	0.0001089305	135.749	190.013	440.132
180000	0.0001066575	134.282	189.213	440.363
190000	0.0001045130	132.874	188.411	440.103
200000	0.0001024406	131.489	187.603	441.101
210000	0.0001003640	130.074	186.778	442.749
220000	0.0000982461	128.617	185.918	444.851
230000	0.0000966963	127.132	185.037	447.126
240000	0.0000939742	125.638	184.143	449.976
250000	0.0000918833	124.110	183.184	453.419
260000	0.0000897488	122.592	182.204	456.887
270000	0.0000881592	121.458	181.444	459.414
280000	0.0000856745	119.635	180.554	460.794
290000	0.0000836985	118.220	179.784	461.408
300000	0.0000818294	116.859	179.064	461.520

Chapter 6

Implementation of PIGNN

6.1 Motivation for GNN

CNN [16] employed mean values in place of unknown values within the 5×5 predictive grid. GNN [40] could fill the same functionality as CNN, whilst replacing this grid with edges only to present nuclei, reducing noise towards edges of the nuclear chart, potentially allowing predictions to be viable within this range.

Information from neighbouring nodes, in this case nuclei, can provide information through odd-even parity, and the linkage between coordinates and mass residual. GNN and CNN are capable of targeted leveraging of information from adjacent nodes, unlike MLP (Multilayer Perceptron).

GNN provides flexibility with respect to which nodes are included, CNN will include nodes in a fixed matrix around a given node. As the nuclear chart is a relatively small dataset, with much of the nuclei lying within range of the edges in terms of the CNN's set matrix size, using a GNN which only selects known neighbours avoids inputting noise through extrapolating unknown nuclei for training.

6.2 Graph Attention Network (GAT)

Pytorch offers `torch_geometric` for GNNs, including GAT.

Utilising a GAT GNN allows for dynamic weighting of neighbouring nodes, rather than via hard-coding as in EKRR, providing extrapolation capabilities of variables such as magic numbers.

With respect to output, GAT has F' output features per head, with K heads (independent attention mechanisms which execute the transformation). Therefore, the output from each GAT layer will consist of KF' features.

When not using GAT, the K parameter is not utilised in and of itself, rather it will be a multiplier for F' (`F_prime`), enabling the Graph Convolutional Network (GCN) implementation to compensate for the heads on each layer of GAT.

6.3 Modelling

For implementation of PIGNN, 3 layers were used, the same number as was used for FCNN to improve comparability.

Utilising `torch.nn.functional.dropout` degraded interpolation and extrapolation results. The test was carried out with parameter `p` set to 0.1 (default is 0.5), therefore only implementing a 10% probability that a given element is dropped out. Had the degraded accuracy in extrapolation been due to overfitting, this mechanism should have mitigated against it, whilst the opposite was observed, therefore implying the degradation in results going to extrapolation over interpolation is due to a distributional concept shift as nuclei approach the edges of the known nuclei range, rather than an overfitting to the AME2016 training data.

Adding parity with edges was deployed experimentally, as this could have allowed for parity to be added through nuclei with the relative coordinates $[(2,0),$

$(-2,0)$, $(0,2)$, $(0,-2)$], without increasing the feature count. However, this negatively impacted RMS of both extrapolation and interpolation, therefore was not utilised in the final model. As was used in FCNN, `fcnn_encoded_nucleus_parity`, an encoded feature reflecting the parity of both N and Z was tested, enabling both to be introduced through a single feature rather than a feature each, although this does raise an issue of making a non-ordinal pattern ordinal, by using a numerical method to symbolise the four possible combinations of N and Z parity. Ablation testing demonstrated separate *Z* and *N* parity features were the most beneficial method of conveying parity to the model.

Various optimiser solutions were tested, as was done with FCNN [3]. SGD was tested for in both FCNN and PIGNN, however was consistently outperformed by Adaptive Moment Estimation (Adam) on both. Whilst SGD was not utilised in the original FCNN implementation [3], it was tested for FCNN and PIGNN to provide each with the same tuning attention. Variations of Adam were tested, with stepped learning rates, schedulers, and variations of the optimiser itself, with Adaptive Moment Estimation with Decoupled Weight Decay Regularization (AdamW) being tested.

```

1  def geometric_data_conversion(dataset, selected_features):
2      dataset = dataset.reset_index(drop=True)
3
4      source_nodes, target_nodes = [], []
5      surrounding_nuclei = [(1,0), (-1,0), (0,1), (0,-1)]
6
7      for nucleus_index, row in dataset.iterrows():
8          for dZ, dN in surrounding_nuclei:
9              close_nuclei = dataset[(dataset["Z"] == row["Z"] + dZ) &
10                                     (dataset["N"] == row["N"] + dN)]
11              if len(close_nuclei) > 0:
12                  source_nodes.append(nucleus_index)

```

```

12         target_nodes.append(close_nuclei.index[0])
13
14         edge_index = torch.tensor([source_nodes, target_nodes], dtype
15         =torch.long)
16
17         x = torch.tensor(dataset[selected_features].values, dtype=
18         torch.float)
19
20         return Data(x=x, edge_index=edge_index)
21
22 class GNN(torch.nn.Module):
23     def __init__(self, initial_in_channels, F_prime, K,
24     final_out_channels, activation_func="gelu", dropout_test=False
25     , GAT=True):
26         """
27         Utilising 3 layers, the same amount as was used for FCNN.
28         When not using GAT, the K parameter is not utilised in and
29         of itself, rather it will be a multiplier for F_prime,
30         enabling the GCN implementation to compensate for the heads on
31         each layer of GAT.
32         """
33         super().__init__()
34         self.dropout_test = dropout_test
35         self.activation_func = activation_func.lower()
36         if GAT:
37             self.layer1 = GATConv(in_channels=initial_in_channels,
38             out_channels=F_prime, heads=K)
39             self.layer2 = GATConv(in_channels=K * F_prime,
40             out_channels=F_prime, heads=K)
41             self.layer3 = GATConv(in_channels=K * F_prime,
42             out_channels=final_out_channels)
43         else:
44             F_prime *= K # Align with the dimensions of head and

```

```

F_prime of GAT
35         self.layer1 = GCNConv(in_channels=initial_in_channels,
out_channels=F_prime)
36         self.layer2 = GCNConv(in_channels=F_prime, out_channels=
F_prime)
37         self.layer3 = GCNConv(in_channels=F_prime, out_channels=
final_out_channels)
38
39     def forward(self, x, edge_index):
40         """Using GELU as was employed for FCNN."""
41         activation_function = F.gelu if self.activation_func == "
gelu" else F.relu
42         x = activation_function(self.layer1(x, edge_index))
43         if self.dropout_test:
44             x = F.dropout(x, p=0.1, training=self.training)
45         x = activation_function(self.layer2(x, edge_index))
46         if self.dropout_test:
47             x = F.dropout(x, p=0.1, training=self.training)
48         x = self.layer3(x, edge_index)
49         return x
50
51
52 class PIGNN:
53     def __init__(
54         self,
55         selected_features,
56         theoretical_model="ws4",
57         N_lower_limit=20,
58         Z_lower_limit=28,
59         N_upper_limit=1000,
60         Z_upper_limit=1000,
61         dropout_test=False,
62         GAT=True

```

```

63     ):
64         self.selected_features = selected_features
65         self.theoretical_model = theoretical_model
66         self.N_lower_limit = N_lower_limit
67         self.Z_lower_limit = Z_lower_limit
68         self.N_upper_limit = N_upper_limit
69         self.Z_upper_limit = Z_upper_limit
70         self.dropout_test = dropout_test
71         self.GAT = GAT
72
73     def get_name(self):
74         return "PIGNN"
75
76     def fit(self, dataset=ame2016_with_features, activation_func=
77 "GELU", optimisation_method="Adam"):
78         torch.manual_seed(0)
79
80         dataset = dataset[(dataset["Z"] > self.Z_lower_limit) &
81 (dataset["N"] > self.N_lower_limit) & (dataset["Z"] <= self.
82 Z_upper_limit) & (dataset["N"] <= self.N_upper_limit)]
83
84         geometric_data = geometric_data_conversion(
85             dataset,
86             selected_features = self.selected_features
87         )
88
89         self.activation_func = activation_func.lower()
90         if self.activation_func != "gelu" and self.activation_func
91 != "relu":
92             raise Exception("Invalid activation function")
93
94         self.model = GNN(
95             initial_in_channels=len(self.selected_features),

```

```

92     F_prime=30, # TO DO: Tune
93     K=4, # TO DO: Tune
94     final_out_channels=1, # TO DO: Tune
95     activation_func=self.activation_func,
96     dropout_test=self.dropout_test,
97     GAT=self.GAT
98 )
99
100     criterion = nn.HuberLoss() # Defaults to delta = 1.0
101
102     optimizer = torch.optim.AdamW(self.model.parameters(), lr=1
103 e-3)
104
105     for epoch in range(50000):
106
107         optimizer.zero_grad()
108         prediction = self.model(geometric_data.x, geometric_data.
109 edge_index) # Forward
110
111         loss = criterion(
112             prediction.squeeze(), # squeeze for same input and
113 target shape, via removing dimension of size 1.
114             torch.tensor(dataset[self.theoretical_model + "
115 _M_residual"].values, dtype=torch.float)
116         )
117
118         loss.backward()
119         optimizer.step()
120
121         if epoch % 100 == 0:
122             if optimisation_method == "Adam" and loss.item() < 0.2:
123                 optimizer = torch.optim.Adam(self.model.parameters(),
124 lr=1e-4) # Resets the optimiser, rather than only adjusting

```



```

the learning rate.
120         optimisation_method = "Adam at reduced learning rate:
1e-4"
121         elif optimisation_method == "Adam at reduced learning
rate: 1e-4" and loss.item() < 0.06:
122             optimizer = torch.optim.Adam(self.model.parameters(),
lr=1e-5)
123             optimisation_method = "Adam at reduced learning rate:
1e-5"
124
125
126     def predict(self, test_dataset):
127         test_dataset = test_dataset[(test_dataset["Z"] > self.
Z_lower_limit) & (test_dataset["N"] > self.N_lower_limit) & (
test_dataset["Z"] <= self.Z_upper_limit) & (test_dataset["N"]
<= self.N_upper_limit)]
128
129
130         geometric_data = geometric_data_conversion(
131             test_dataset,
132             selected_features = self.selected_features
133         )
134
135         with torch.no_grad():
136             predicted_mass_residuals = self.model(geometric_data.x,
geometric_data.edge_index)
137
138         return predicted_mass_residuals.squeeze().numpy()
139
140
141

```

6.4 Feature Selection

A baseline feature selection utilised the same features as FCNN [3]. As different models may prefer a different set of features, utilising the same features across models does not necessarily provide a fair comparison. Whilst this was initially effective, after more rigorous selection, considering the physics represented via a given feature alongside ablations to test said features' improvement enabled a more suitable feature set for GNN to be selected.

Ablations were carried out to discover which features should be utilised, as was done for parity when selecting between not including it, using edges to represent, adding via a single encoded feature or adding via two features, one for Z and one for N . Ultimately, the parity implementation testing yielded two features to be selected, Z parity and N parity. Using ablations allowed for performance changes to be quantified, ensuring a given feature provided meaningful improvement or could be omitted from the selection.

Shell correction of nucleus and deformation energy of nucleus were considered. Both can incorporate meaningful physics into the model, however, upon ablation testing they degraded the overall performance when tested.

Z and N , two standard features of other ML [1, 3] approaches actually degraded also. As GNN implements the Z and N coordinates on the nuclear chart as edges, the information introduced from Z and N themselves is overlapping with the fitting derived from said edges. Therefore, Z and N are not currently included in the GNN feature selection.

$|Z - m|$ and $|N - m|$ provide distance to the nearest magic number from the given proton and neutron number respectively. Upon ablation testing these values did provide a meaningful contribution to performance.

The proton and neutron shell numbers were also implemented, along with the neutron excess calculated via $N - Z$. All three of these features were tested via

ablations.

β_2 of WS4 was selected as a feature. WS4 β_4 and β_6 were also considered, they did not perform as effectively as β_2 , and incorporation of multiple β features simultaneously caused a degradation in performance as reflected via worsened RMS.

Using the same model for the theoretical base model, such as WS4, and as an input feature of said model allows for a coupling to occur between the two. The feature enables the ML model to predict the residual based on the internal workings of the theoretical model. Swapping the base theoretical model, and the feature's corresponding theoretical model can be carried out in order to evaluate this concept.

As WS4 provided the strongest theoretical model for refinement when tested with GNN, it was selected as the primary theoretical model, with others included for reference. Due to WS4 already being included as the theoretical model for refinement, an approach using a feature from another theoretical model was evaluated. FRDM provided a relatively effective theoretical model, therefore incorporating a feature from FRDM could have allowed for both WS4 and FRDM to be utilised in predictions. However, upon ablation testing, β_2 of WS4 outperformed β_2 of FRDM including with WS4 as the theoretical base, therefore β_2 of WS4 was selected over another theoretical model's feature.

Note, multiple versions of certain theoretical models are available, such as WS and WS4 or DZ-10 and DZ-28, for FRDM there is a 1995 version and a 2012 version, the features tested were those of the 1995 FRDM.

The features selected after ablations were:

- $|Z - m|$ and $|N - m|$, distance to the nearest magic number from the given proton and neutron number respectively.
- Proton and neutron shell numbers

- Z parity and N parity
- Neutron excess ($N - Z$)
- β_2 of WS4

Chapter 7

Results

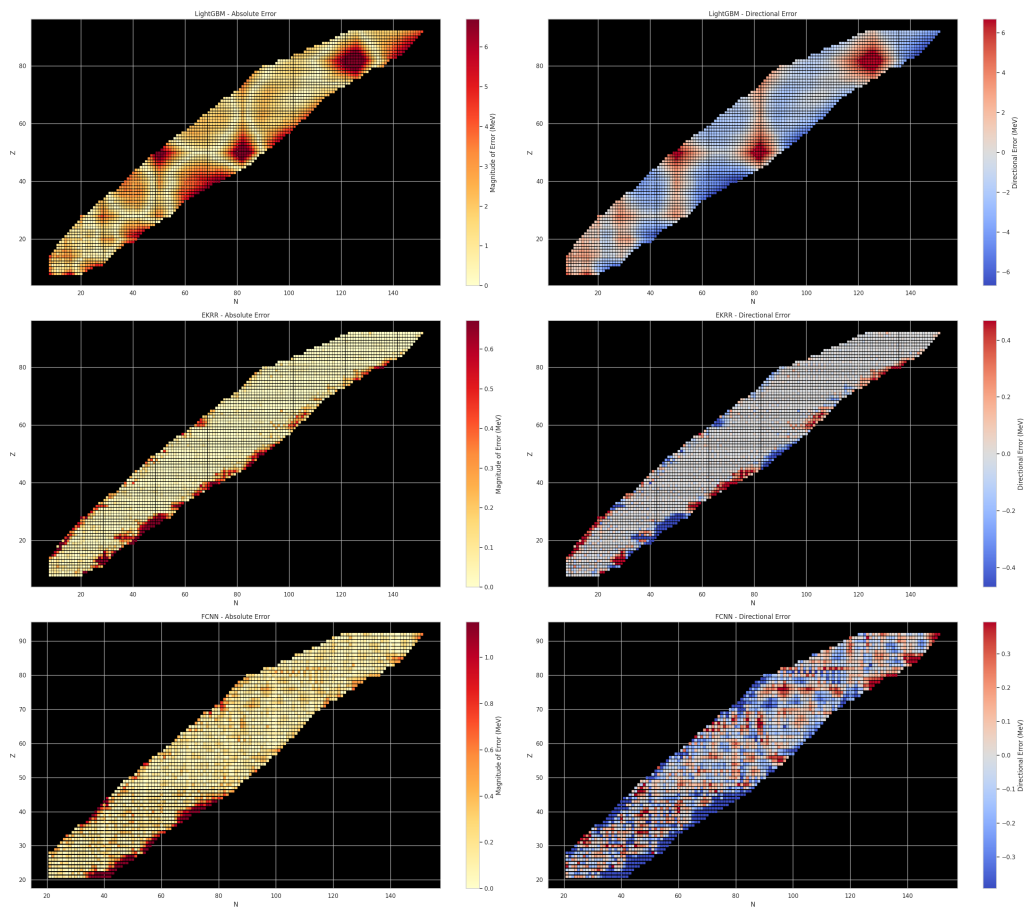


Figure 7.1: Nuclear chart heatmaps of reengineered existing state-of-the-art models

Utilising a heatmap of the nuclear chart enabled insight into how effectively the model was performing in any region. Patterns observed displayed physics-based patterns, and regions of higher error rates.

Table 7.1: RMS (keV) Achieved by ML refinement

Model	Implementation	Z and N Limits	Training RMS	Interpolation RMS	Extrapolation RMS	66 nuclei of LightGBM
WS4	WS4 without ML	$7 < N, 7 < Z$	466.038	413.820	1288.145	–
WS4	WS4 without ML	$20 < N, 20 < Z$	447.961	401.575	1288.145	–
PIGNN	WS4, 20,000 epochs	$20 < N, 28 < Z$	282.846	248.246	497.839	–
PIGNN	WS4, 50,000 epochs	$20 < N, 28 < Z$	218.796	211.227	404.736	–
FCNN	WS4, Modified Case B Method III	$20 < N, 20 < Z$	138.993	191.766	437.747	–
EKRR	WS4	$7 < N, 7 < Z$	30.875	165.951	772.917	–
EKRR	WS4	$7 < N, 7 < Z < 93$	32.345	174.896	519.940	–
LightGBM	LDM	$7 < N, 7 < Z < 93$	203.217	186.727	611.910	563.189
LightGBM	WS4	$7 < N, 7 < Z < 93$	173.772	181.567	568.093	542.282
LightGBM	WS4, full range	$7 < N, 7 < Z$	180.388	175.069	475.397	527.422

All four models, LightGBM, EKRR, FCNN and PIGNN were able to achieve RMS outperforming WS4 without ML therefore successfully providing ML refinement. WS4 tends to gain the least improvement from ML [1], despite this it does consistently maintain the lowest RMS after refinement, this is due to the theoretical model itself already being relatively more accurate than other such models.

WS4 consistently provided lower RMS than other theoretical models, both in the original results [1], as well as those of models created.

The results in table 7.1 are of a singular run, which do not account for distribution. One model achieving better metrics does not prove it outperforms a lower scoring model. Therefore, despite PIGNN returning the lowest RMS for extrapolation, whether PIGNN is outperforming or within the performance range of existing models cannot be confirmed. To maintain consistency, a 0 seed was used when fitting throughout. As model tuning is ongoing, carrying out a larger quantity of runs and analysing the distribution has yet to be carried out, whilst EKRR and LightGBM implementations are relatively quick, FCNN and PIGNN are time consuming especially when numerous runs are required. A distribution analysis will be carried out once tuning is complete, or near thereto.

Z and N limits were varied, FCNN and PIGNN performed more effectively with heavier nuclei, although all models benefitted from removing nuclei lighter than oxygen.

Chapter 8

Research Questions

Although implementation of the four models: LightGBM, EKRR, FCNN and PIGNN has been carried out, there is still further room for exploration. The following are research questions for said ongoing exploration.

RQ1: *How does PIGNN compare with the reengineered existing state-of-the-art solutions?*

PIGNN appears comparable from the results observed, including those in the results chapter 7. However, a comparison of performance distribution is yet to be carried out, this will allow for both a comparison of performance between models, as well as quantification of confidence in performance via confidence intervals.

RQ2: *Would an ensemble of ML refinement models be viable?*

As the preferable model may vary between region, an ensemble technique could be employed which utilises one model for a range best suited to the given model, and a secondary model for the range to which it is best suited.

RQ3: *Would chaining ML refinement models improve predictions?*

Chaining methods could also be explored. The models output predictions as a

dataset of the nuclear chart, as was shown in the heatmaps. Via inputting this predicted dataset into another model a pipeline could be tested with multiple models. As the models have already been reengineered, the initial pipeline could be readily created, however additional tuning will likely be required.

RQ4: *How do predictions degrade as extrapolation distance increases?*

As a predicted nucleus goes further from the trained nuclei it becomes more difficult to predict, both due to its increased distance from the training data and from these regions also being unstable. Analysis could be carried out to explore the trend for increased residual as the distance from the AME2016 known nuclei increases. The data for this research question is available as the models already output their prediction of each nucleus, therefore analysis on said data should be able to answer this research question.

Chapter 9

Conclusion

The research carried out provides contributions on two fronts:

- The novel PIGNN model proposed.
- A comparison between reengineered existing state-of-the-art models, with a consistent task.

9.1 Progress

All models were reengineered from available information, and retuned for their given task. This ensured that the new model would not gain relatively better performance merely due to being tuned for the task to which it is being applied, whilst other models remained tuned for a different task. Tuning all models also mitigated the risk of degraded performance due to incomplete reimplementation from missing information as the original code was not discovered.

9.2 Future work

PIGNN is providing competitive results to the current implementations of LightGBM, EKRR and FCNN in the exotic nuclei areas of the nuclear chart. As EKRR [2] is a closed form solution, tuning opportunities are more limited than for LightGBM, FCNN and PIGNN, however, input nuclei selection has been tested for EKRR, with substantial changes in performance observed for it, along with the other three models. As the project is yet to be completed, work will continue in attempting to tune PIGNN, with the same tuning options explored for FCNN. LightGBM has also been tested for potential tuning improvements, however, the tuning improvements to PIGNN are not as transferable to LightGBM as they are to FCNN, due to FCNN also being a NN.

As model performance varies inconsistently across regions, with the best performing model changing depending upon the subset of the nuclear chart used, it could be possible to create a hybrid approach, using EKRR or LightGBM for lighter nuclei, and FCNN or PIGNN which performed more effectively on heavier nuclei.

References

- [1] Z.-P. Gao, Y.-J. Wang, H.-L. Lü, Q.-F. Li, C.-W. Shen, and L. Liu, “Machine learning the nuclear mass,” *Nuclear Science and Techniques*, vol. 32, no. 10, p. 109, 2021. ii, 1, 2, 5, 6, 8, 9, 11, 12, 13, 14, 15, 16, 17, 19, 21, 22, 23, 28, 56, 61
- [2] X. Wu, L. Guo, and P. Zhao, “Nuclear masses in extended kernel ridge regression with odd-even effects,” *Physics Letters B*, vol. 819, p. 136387, 2021. 4, 12, 13, 28, 32, 66
- [3] Y. Huang, J. Chen, J. Jia, L.-M. Liu, Y.-G. Ma, and C. Zhang, “Validation and extrapolation of atomic masses with a physics-informed fully connected neural network,” *Physical Review C*, vol. 111, no. 3, p. 034329, 2025. ii, 1, 2, 5, 9, 11, 12, 13, 14, 15, 28, 36, 39, 40, 50, 56
- [4] L. Tang and Z.-H. Zhang, “Nuclear charge radius predictions by kernel ridge regression with odd-even effects,” *Nuclear Science and Techniques*, vol. 35, no. 2, 2024. ii, 4, 5, 12
- [5] Y. Lin, J.-X. Li, and H.-F. Zhang, “Transfer learning and neural networks in predicting quadrupole deformation,” *Chinese Physics C*, vol. 48, no. 6, p. 064106, 2024. 11, 15
- [6] B. Canbula, “Predicting nuclear level density using a physics-informed neural

REFERENCES

- network with multi-task learning,” *Applied Sciences*, vol. 16, no. 1, p. 312, 2026. ii, 4, 5, 12
- [7] C. Ma, M. Bao, Z. Niu, Y. Zhao, and A. Arima, “New extrapolation method for predicting nuclear masses,” *Physical Review C*, vol. 101, no. 4, p. 045204, 2020. vii, 5, 9, 10, 15
- [8] D. Lunney, J. Pearson, and C. Thibault, “Recent trends in the determination of nuclear masses,” *Reviews of Modern Physics*, vol. 75, no. 3, p. 1021, 2003. 1, 5, 6
- [9] C. v. Weizsäcker, “Zur Theorie der Kernmassen,” *Zeitschrift für Physik*, vol. 96, pp. 431–458, 1935. 1, 4, 5, 6, 17
- [10] G. Garvey, W. Gerace, R. Jaffe, I. Talmi, and I. Kelson, “Set of nuclear-mass relations and a resultant mass table,” *Reviews of Modern Physics*, vol. 41, p. S1, 1969. 7
- [11] J. Duflo and A. Zuker, “Microscopic mass formulas,” *Physical Review C*, vol. 52, no. 1, p. R23, 1995. 6
- [12] N. Wang, M. Liu, X. Wu, and J. Meng, “Surface diffuseness correction in global mass formula,” *Physics Letters B*, vol. 734, pp. 215–219, 2014. 6, 7, 8
- [13] P. Möller, A. J. Sierk, T. Ichikawa, and H. Sagawa, “Nuclear ground-state masses and deformations: FRDM (2012),” *Atomic Data and Nuclear Data Tables*, vol. 109, pp. 1–204, 2016. 1, 5, 6
- [14] Z. Niu and H. Liang, “Nuclear mass predictions based on Bayesian neural network approach with pairing and shell effects,” *Physics Letters B*, vol. 778, pp. 48–53, 2018. 1, 2, 5, 8, 9, 11, 13, 15

REFERENCES

- [15] M. R. Mumpower, R. Surman, G. McLaughlin, and A. Aprahamian, “The impact of individual nuclear properties on r-process nucleosynthesis,” *Progress in Particle and Nuclear Physics*, vol. 86, pp. 86–126, 2016. 1, 5
- [16] Y. Lu, T. Shang, P. Du, J. Li, H. Liang, and Z. Niu, “Nuclear mass predictions based on a convolutional neural network,” *Physical Review C*, vol. 111, no. 1, p. 014325, 2025. 1, 5, 6, 9, 14, 15, 48
- [17] A. Wapstra, G. Audi, and C. Thibault, “The AME2003 atomic mass evaluation (I). Evaluation of input data, adjustment procedures,” *Nuclear Physics A*, vol. 729, no. 1, pp. 129–336, 2003. 2, 5, 11, 12, 15
- [18] G. Audi, A. Wapstra, and C. Thibault, “The AME2003 atomic mass evaluation (II). Tables, graphs and references,” *Nuclear Physics A*, vol. 729, no. 1, pp. 337–676, 2003.
- [19] G. Audi, M. Wang, A. Wapstra, F. Kondev, M. MacCormick, X. Xu, and B. Pfeiffer, “The AME2012 atomic mass evaluation,” *Chinese Physics C*, vol. 36, no. 12, pp. 1287–1602, 2012.
- [20] M. Wang, G. Audi, A. Wapstra, F. Kondev, M. MacCormick, X. Xu, and B. Pfeiffer, “The AME2012 atomic mass evaluation,” *Chinese Physics C*, vol. 36, no. 12, pp. 1603–2014, 2012.
- [21] W. Huang, G. Audi, M. Wang, F. Kondev, S. Naimi, and X. Xu, “The AME2016 atomic mass evaluation (I). Evaluation of input data; and adjustment procedures,” *Chinese Physics C*, vol. 41, no. 3, p. 030002, 2017. 2, 13, 21
- [22] M. Wang, G. Audi, F. Kondev, W. Huang, S. Naimi, and X. Xu, “The AME2016 atomic mass evaluation (II). Tables, graphs and references,” *Chinese Physics C*, vol. 41, no. 3, p. 030003, 2017. 2, 13

REFERENCES

- [23] W. Huang, M. Wang, F. G. Kondev, G. Audi, and S. Naimi, “The AME 2020 atomic mass evaluation (I). Evaluation of input data, and adjustment procedures,” *Chinese Physics C*, vol. 45, no. 3, p. 030002, 2021. 2, 13
- [24] M. Wang, W. J. Huang, F. G. Kondev, G. Audi, and S. Naimi, “The AME 2020 atomic mass evaluation (II). Tables, graphs and references,” *Chinese Physics C*, vol. 45, no. 3, p. 030003, 2021. 2, 5, 11, 12, 13, 15, 21
- [25] S. Qu, J.-Y. Zhang, and M. Bao, “Nuclear mass predictions with a Bayesian neural network,” *Chinese Physics C*, vol. 49, no. 10, p. 104106, 2025. 2, 5, 8, 9, 10, 11, 15
- [26] E. Yüksel, D. Soydaner, and H. Bahtiyar, “Nuclear mass predictions using machine learning models,” *Physical Review C*, vol. 109, no. 6, p. 064322, 2024. 6, 7, 8, 10, 11, 12, 13, 14, 15
- [27] Z. Niu, J. Fang, and Y. Niu, “Comparative study of radial basis function and Bayesian neural network approaches in nuclear mass predictions,” *Physical Review C*, vol. 100, no. 5, p. 054311, 2019. 5, 8, 10, 11, 12
- [28] L. Neufcourt, Y. Cao, W. Nazarewicz, and F. Viens, “Bayesian approach to model-based extrapolation of nuclear observables,” *Physical Review C*, vol. 98, no. 3, p. 034318, 2018. 5, 9, 14
- [29] L. Neufcourt, Y. Cao, S. A. Giuliani, W. Nazarewicz, E. Olsen, and O. B. Tarasov, “Quantified limits of the nuclear landscape,” *Physical Review C*, vol. 101, no. 4, p. 044307, 2020. 9, 11, 15
- [30] A. E. Lovell, A. T. Mohan, T. M. Sprouse, and M. R. Mumpower, “Nuclear masses learned from a probabilistic neural network,” *Physical Review C*, vol. 106, no. 1, p. 014305, 2022. 5, 9, 13, 15

REFERENCES

- [31] M. R. Mumpower, T. M. Sprouse, A. E. Lovell, and A. T. Mohan, “Physically interpretable machine learning for nuclear masses,” *Physical Review C*, vol. 106, no. 2, p. L021301, 2022. 2, 5, 14, 15
- [32] X. Wu and P. Zhao, “Predicting nuclear masses with the kernel ridge regression,” *Physical Review C*, vol. 101, no. 5, p. 051301, 2020. 13
- [33] Z. Niu and H. Liang, “Nuclear mass predictions with machine learning reaching the accuracy required by r-process studies,” *Physical Review C*, vol. 106, no. 2, p. L021303, 2022. 5, 15
- [34] X.-C. Ming, H.-F. Zhang, R.-R. Xu, X.-D. Sun, Y. Tian, and Z.-G. Ge, “Nuclear mass based on the multi-task learning neural network method,” *Nuclear Science and Techniques*, vol. 33, no. 4, p. 48, 2022. 5, 9, 11, 12, 15
- [35] N. Wang, M. Liu, and X. Wu, “Modification of nuclear mass formula by considering isospin effects,” *Physical Review C—Nuclear Physics*, vol. 81, no. 4, p. 044322, 2010. 5, 6
- [36] N. Wang, Z. Liang, M. Liu, and X. Wu, “Mirror nuclei constraint in nuclear mass formula,” *Physical Review C—Nuclear Physics*, vol. 82, no. 4, p. 044304, 2010. 6, 38
- [37] M. Liu, N. Wang, Y. Deng, and X. Wu, “Further improvements on a global nuclear mass model,” *Physical Review C—Nuclear Physics*, vol. 84, no. 1, p. 014333, 2011. 6
- [38] I. Bentley, A. Fiorito III, M. Gebran, W. Porter, and A. Aprahamian, “Combined Garvey Kelson Relations for Mass Determinations and Machine Learning,” *arXiv preprint arXiv:2603.07015*, 2026. 5, 7, 9, 15

REFERENCES

- [39] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019. 5, 10, 12
- [40] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, “The graph neural network model,” *IEEE transactions on neural networks*, vol. 20, no. 1, pp. 61–80, 2009. 14, 48
- [41] R. Caruana, “Multitask learning,” *Machine learning*, vol. 28, no. 1, pp. 41–75, 1997. 11
- [42] S. J. Pan and Q. Yang, “A survey on transfer learning,” *IEEE Transactions on knowledge and data engineering*, vol. 22, no. 10, pp. 1345–1359, 2010. 5, 11
- [43] M. Bao, H. Jiang, and Y.-M. Zhao, “Predictions.xlsx,” <https://cstr.cn/31253.11.sciencedb.j00177.00003.00876FCE>, 2023, v2. Science Data Bank [DS/OL]. Accessed: 2026-06-03. 5, 15
- [44] A. Boehnlein, M. Diefenthaler, N. Sato, M. Schram, V. Ziegler, C. Fanelli, M. Hjorth-Jensen, T. Horn, M. P. Kuchera, D. Lee, W. Nazarewicz, P. Ostroumov, K. Orginos, A. Poon, X.-N. Wang, A. Scheinker, M. S. Smith, and L.-G. Pang, “Colloquium: Machine learning in nuclear physics,” *Reviews of Modern Physics*, vol. 94, no. 3, p. 031003, 2022. 5, 7
- [45] W. He, Q. Li, Y. Ma, Z. Niu, J. Pei, and Y. Zhang, “Machine learning in nuclear physics at low and intermediate energies,” *Science China Physics, Mechanics & Astronomy*, vol. 66, no. 8, p. 282001, 2023. 5, 7
- [46] H. F. Zhang, L. H. Wang, J. P. Yin, P. H. Chen, and H. F. Zhang, “Performance of the levenberg–marquardt neural network approach in nuclear mass

REFERENCES

- prediction,” *Journal of Physics G: Nuclear and Particle Physics*, vol. 44, no. 4, p. 045110, 2017. 16, 17, 19
- [47] P. Roy Chowdhury and D. Basu, “Nuclear matter properties with the re-evaluated coefficients of liquid drop model,” *Acta Phys. Polon. B*, vol. 37, no. arXiv: nucl-th/0408013, pp. 1833–1846, 2006. 16, 17